



Reparaciones y venta de autopartes

DOCUMENTACIÓN

LOS HERMANOS

2.0

AVILA GARCIA EDAHI YAXQUIN
GACHUZ ALLENDE JOSE CARLOS



Objetivo General

Automatizar todos los métodos manuales (hojas de papel, libretas, archivos físicos) por un sistema web que digitalice, centralice y almacene toda la información de inventario, servicios, reportes del taller mecánico. Así mismo se estará eliminando el desorden físico y el riesgo de pérdida de documentos para evitar la pérdida de datos.

Problemática

El taller mecánico “Los Hermanos” es un negocio familiar de reparación de automóviles y la venta de auto-partes. Sin embargo, su problema es por el uso de métodos manuales y para gestionar sus procesos:

- **Ineficiente y propenso a errores**

El inventario de refacciones se realiza manualmente en hojas de papel. Este proceso es extremadamente lento, posibles errores humanos (como anotaciones incorrectas o cálculos mal hechos) y no permite tener una visión en tiempo real del stock disponible.

- **Complejo**

El registro de ventas diarias se lleva en hojas cuadriculadas que luego se archivan en carpetas organizadas por mes.

- **Gestión de Clientes y Historial de Servicios Limitada**

Aunque se entrega una hoja de remisión al cliente, este historial queda físicamente archivado en una carpeta.



- **Problemas de Espacio y Organización Física**

La oficina está "repleta" de carpetas y archivos. Este desorden ocupa un espacio valioso que podría ser utilizado para otras actividades.

Solución al problema

Para resolver los problemas de gestión manual, se propone el desarrollo de un sistema web. Esta solución digitalizará y centralizará todas las operaciones, eliminando el uso de libretas y hojas de papel, lo que resultará en mayor eficiencia y control. Esta solución se basa en módulos.

Módulo de Facturación

- Generación Automatizada de Tickets: Por cada compra o servicio, el sistema generará un ticket que incluirá todos los detalles.
- Registro Inmediato: Cada venta quedará registrada al instante en la base de datos, asociada al cliente y al artículo o servicio.
- Cálculo Automático: El sistema calculará automáticamente el total, el IVA y cualquier otro impuesto o descuento, eliminando errores de cálculo manual.

Módulo de Inventario

- Registro de Refacciones: Se podrán dar de alta nuevas refacciones en el sistema con todos sus atributos: Nombre, Categoría, Marca compatible, Modelo de auto compatible, Precio de compra, Precio de venta, Fecha de publicación.
- Actualizaciones en Tiempo Real: El stock se actualizará automáticamente cada vez que se realice una venta o una compra de artículos.



Módulo de Reportes

- Registro Completo de Ventas: Toda la información de ventas se almacenará digitalmente, permitiendo generar reportes detallados por día, semana, mes o rango de fechas personalizado.
- Análisis de Datos: Los reportes permitirán identificar fácilmente los productos y servicios más vendidos, los ingresos generados y las tendencias de venta, facilitando la toma de decisiones estratégicas.

Módulo de servicios

- Historial de Clientes: El sistema almacenará los datos de los clientes y el historial completo de compras y servicios realizados en sus vehículos. Al ingresar un cliente, se podrá ver rápidamente qué servicios se le han hecho antes.
- Compatibilidad con Vehículos: Al registrar una venta o servicio, se asociará al modelo específico del automóvil del cliente, enriqueciendo la base de datos y el historial.

Esta solución no solo automatiza las tareas actuales, sino que transforma la información del taller en una herramienta estratégica para su crecimiento.

DISEÑO DE SOFTWARE QUE SE PROPONE

Arquitectura a usar: micro-servicios

Justificación

El uso de esta arquitectura se justifica en que cada integrante domina un lenguaje específico, junto con sus bibliotecas y mejores prácticas. Esto permitirá ahorrar tiempo en el desarrollo, ya que se evitará la capacitación sobre sintaxis, convenciones y recomendaciones propias de cada tecnología. Como punto adicional, en la parte del lenguaje de programación se debe al poco tiempo que tenemos para el desarrollo. Hasta windows esta hecho con varios lenguajes, por la entrega continua que maneja microsoft.



Aplicación

- **Ventas:** Se propone que su función permita que el modulo de servicios pueda interactuar con el micro-servicio cuando haya una venta. De esta manera se evitara que se le tenga que dar aviso al modulo de inventario. En pocas palabras la parte de servicios actualizara el stock cada vez que haya una venta. Y así mismo se actualizara en el inventario.

Bibliotecas y lenguaje de programación para el desarrollo del microservicio

- **Typescript:** TypeScript es un lenguaje de programación fuertemente tipado que se basa en JavaScript, lo cual hace que sigamos buenas practicas de codificación. Igual siendo un lenguaje de alto nivel, nos proporcionara un rapido desarrollo.
- **Express:** Con una gran cantidad de métodos de utilidad HTTP y middleware a su disposición, crear el microservicio sera rápido desarrollar debido al poco tiempo que tenemos.
- **Sequelize:** Es un moderno TypeScript y Node.js ORM para Oracle, Postgres, MySQL, MariaDB, SQLite y SQL Server, y más. Y así mismo aseguramos una codificación limpia y se podrá mejorar la seguridad para evitar las inyecciones de SQL.

Finalmente, siguiendo con las buenas practicas de codificación, se ocupara el patrón de diseño llamado Builder.

Metodología a usar: Kanban

Justificación

Se eligió Kanban porque, en caso de surgir problemas durante la implementación de funciones, el equipo podrá reportarlos oportunamente y tomar las medidas necesarias para resolverlos. Asimismo, se adoptó esta metodología para el desarrollo del microservicio, ya



que es fundamental mantener una comunicación constante sobre cómo debe consumirse y cómo debe implementarse la infraestructura para integrarlo correctamente en el sitio web, evitando así retrasos. Además, se considera que cada tarea debe tener un límite de tiempo para realizar pruebas que garanticen su correcto funcionamiento. Finalmente, también se eligió Laravel por estar basado en el patrón MVC, cuyas ventajas incluyen una mejor organización del trabajo en equipo, permitiendo asignar funcionalidades específicas a cada integrante.

Para administrar el tiempo de kanban se va a ocupar Asana

Aplicación

El tablero se basará en 4 columnas:

1. En consulta: En esta fase se consultará con todo el equipo cómo se implementará la solución, qué convenciones se adoptarán y se tomará una decisión conjunta. También se evaluará si existen malas prácticas de codificación que deban corregirse, etc.
2. En desarrollo: Se desarrollará la función con base en la retroalimentación obtenida durante la fase de consulta
3. En implementación: Se implementará la función desarrollada en la página web.
4. En testeo: Se probará la función; en esta etapa se confirmará que esté funcionando correctamente en conjunto con las demás funcionalidades.

Colores:

- Blanco.
- Azul.
- Amarillo
- Verde

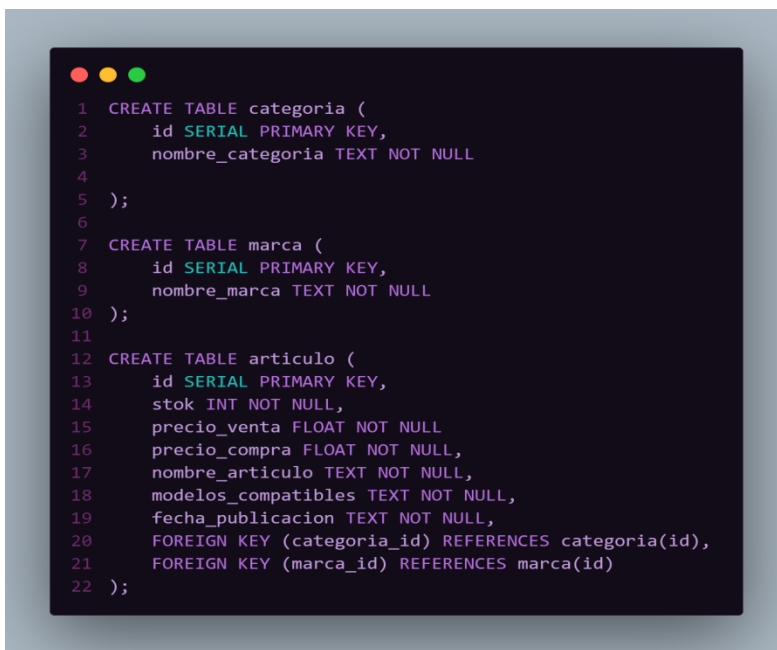
Framework a usar: Laravel

Justificación

Se eligió Laravel porque el equipo está familiarizado con el patrón MVC y con PHP. Además, se cuenta con experiencia en proyectos anteriores, lo cual representa una ventaja al momento de desarrollar el módulo de inventario (Refacciones). También se optó por este framework porque, normalmente, las aplicaciones basadas en MVC se comunican con microservicios. Por otra parte, Laravel es una tecnología madura, lo que permitirá ahorrar tiempo al aprovechar bibliotecas que facilitan la implementación de funcionalidades en el sitio web.

Aplicación

Antes de dar la explicación sobre el uso, se tiene propuesta una estructura de la base de datos. Esta representación ayuda a entender mejor **(Ver figura 1)**:



```
1 CREATE TABLE categoria (  
2     id SERIAL PRIMARY KEY,  
3     nombre_categoria TEXT NOT NULL  
4 )  
5 );  
6  
7 CREATE TABLE marca (  
8     id SERIAL PRIMARY KEY,  
9     nombre_marca TEXT NOT NULL  
10 );  
11  
12 CREATE TABLE articulo (  
13     id SERIAL PRIMARY KEY,  
14     stok INT NOT NULL,  
15     precio_venta FLOAT NOT NULL,  
16     precio_compra FLOAT NOT NULL,  
17     nombre_articulo TEXT NOT NULL,  
18     modelos_compatibles TEXT NOT NULL,  
19     fecha_publicacion TEXT NOT NULL,  
20     FOREIGN KEY (categoria_id) REFERENCES categoria(id),  
21     FOREIGN KEY (marca_id) REFERENCES marca(id)  
22 );
```

Figura 1: Estructura SQL

Modelos

1. Se hará un modelo para la tabla categoría y marca.
2. Se hará un modelo para la tabla articulo que tendrá relación con categoría y marca.

Vistas

3. Inicio: Tendrá contenido relacionado con el taller, de ahí se podrá navegar al inventario.
4. Artículos: Se mostrara todo lo disponible, se podrá filtrar por categoría y marca.
5. Formulario de registro: Se registrara cada dato (Categoría, Marca, Articulo), mediante los atributos que aparecen en la **figura 1**.
6. Formulario de actualización: Se podrá actualizar un dato ya existente (Categoría, Marca, Articulo).

Controladores

Tendrán los métodos necesarios para la administración tomando en cuenta categorías, marcas y artículos.

Nota: Revise el apartado de anexo, es posible que se encuentren datos adicionales, sobre las vistas, modelos y controladores.

Patrón de diseño a usar: Builder

Imagen ilustrativa

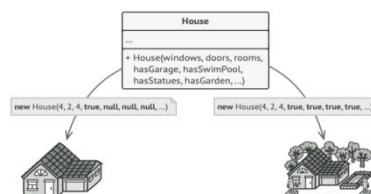


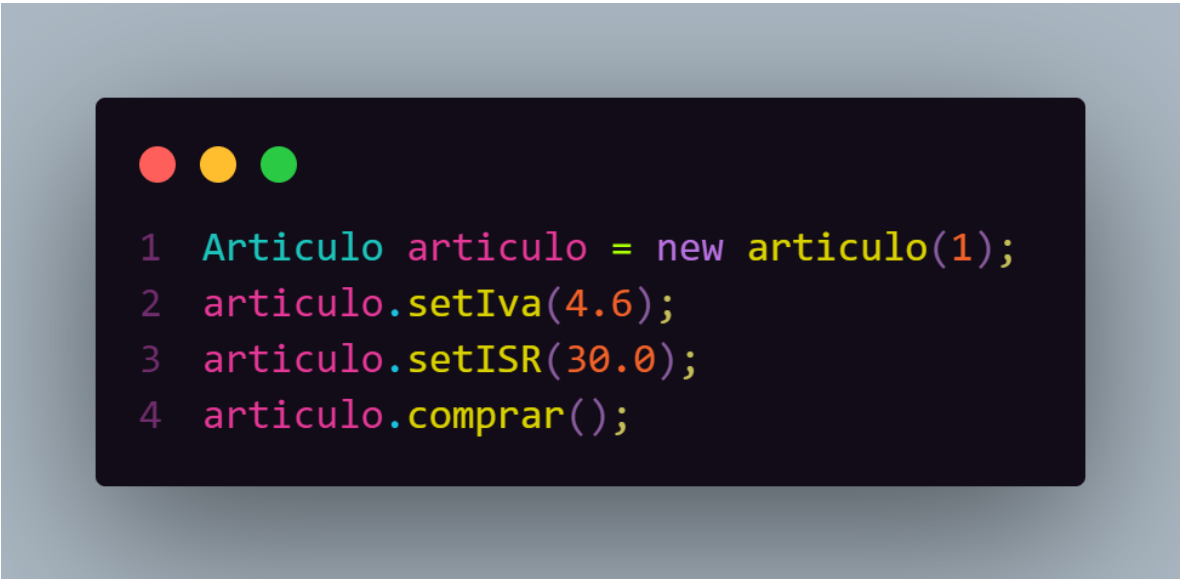
Figura 2: Patrón Builder

Justificación

Se eligió este patrón para evitar esta creando subclases en el micro servicio, haciendo que sea más legible el código, lo que facilitara la estabilidad. Esto en pocas palabras evitara el código espagueti.

Aplicación

Se usara para que el modulo de servicios, calcule los impuestos necesarios, en forma de cadena, ya que todas las compras tiene diferentes porcentajes o diferentes impuestos. En pocas palabras al momento de empezar la compra se le va ir agregando los impuestos necesarios. Un ejemplo seria **(Ver figura 3)**:



```
1  Artículo articulo = new articulo(1);
2  articulo.setIva(4.6);
3  articulo.setISR(30.0);
4  articulo.comprar();
```

Figura 3: Ejemplo de patrón builder aplica a la propuesta

JUSTIFICACIÓN DE LA HERRAMIENTA (GITHUB)

Para el desarrollo del sistema web propuesto para el taller “Los Hermanos”, se ha determinado que la plataforma GitHub será el sistema de control de versiones seleccionado.



Esta decisión se plantea en tres partes van directamente con las necesidades del proyecto y la visión a futuro del negocio:

- Trazabilidad y Control de Actualizaciones:

La justificación principal para la adopción de GitHub radica en su capacidad superior para la trazabilidad. La plataforma permitirá implementar un “apartado de actualizaciones” dedicado y estructurado. En la práctica, esto significa que por cada nueva versión del sistema, el equipo podrá subir un registro claro de los cambios implementados, el número de versión específico, y el código fuente completo asociado a esa versión (por ejemplo, en un archivo .zip). Esta función es crucial, ya que servirá como un historial para consultar los cambios que ha sufrido el software de una versión a otra, garantizando un control histórico completo y facilitando la auditoría del código.

- Escalabilidad y Flexibilidad a Futuro:

Se seleccionó GitHub porque permite tener un “mayor alcance de datos”, lo que se traduce directamente en una mayor flexibilidad para que el proyecto pueda crecer en el futuro. Este aspecto es vital, ya que esta capacidad de expansión ayuda directamente a la refaccionaría si quiere crecer. El sistema no estará limitado por su herramienta de versionado si el negocio decide añadir nuevos módulos, integrarse con otros sistemas o expandir sus operaciones, la plataforma podrá soportar este crecimiento sin fricciones técnicas.

- Integración de Pruebas y Manejo de Errores:

Finalmente, se considera fundamental la capacidad de la plataforma para integrarse con procesos de aseguramiento de calidad. GitHub permite al equipo de desarrollo realizar pruebas y gestionar la detección de errores de manera automatizada mediante el uso de workflows (flujos de trabajo). Esto significa que se pueden configurar rutinas automáticas que verifiquen el código en busca de fallos antes de que este sea implementado, mejorando significativamente la estabilidad y confiabilidad del sistema web que utilizará el taller.

FLUJO DE TRABAJO



Metodología de Ramas y Políticas de Colaboración

Se establecerá una metodología estricta para la gestión de ramas y políticas claras de colaboración para asegurar un desarrollo ordenado, controlado y de alta calidad en el repositorio del proyecto.

Nomenclatura y Estrategia de Ramas

Para organizar el flujo de trabajo de desarrollo, se implementará una estrategia de ramas específica, donde cada rama tiene un propósito definido:

PRINCIPAL

- Prefijo: main (o master)
- Formato: main
- Uso: Contiene siempre el código que está en producción.
- Regla: No deben tener caracteres especiales.

HOTFIX

- Prefijo: hotfix
- Formato: hotfix/<identificador>/<descripción>/<yyyy-mm-dd>
- Uso: Corrección de errores críticos que deben ir directamente a producción.
- Ejemplo: hotfix/LOGIN_01/fallo_autenticacion/2025-10-20
- Regla: No deben tener caracteres especiales.

FEATURE

- Prefijo: feature
- Formato: feature/<nombre_feature>/<descripción>/<yyyy-mm-dd>
- Uso: Desarrollo de nuevas funcionalidades o mejoras temporales.



- Ejemplo: hotfix/LOGIN_01/fallo_autenticacion/2025-10-20
- Regla: No deben tener caracteres especiales.

DEVELOP

- Prefijo: develop
- Formato: develop
- Uso: Integra de forma continua todas las features que aún están en fase de pruebas.
- Regla: No deben tener caracteres especiales.

RELEASE

- Prefijo: release
- Formato: release/<versión>/<yyyy-mm-dd>
- Uso: Preparación de una versión final para su despliegue.
- Ejemplo: release/v1.2.0/2025-10-20
- Regla: No deben tener caracteres especiales.

ACCESO DE USUARIOS.

Los permisos de acceso al repositorio se gestionarán de forma granular para mantener la seguridad y el control.

POLÍTICAS DE COMBINACIÓN DE RAMAS

Requerir solicitudes de extracción

- Quién establece la regla: Administrador del repositorio (o usuario con permiso Maintain).
- Quién la cumple: cualquier miembro del equipo que quiera integrar cambios, debe crear una pull-request; sin ella no se puede fusionar.

Requerir revisiones



- Quién define cuántas aprobaciones son necesarias: Administrador (o quien tenga permiso Maintain).
- Quién aprueba: los revisores asignados, que deben tener permiso Write o superior y marcar “Approve” en la PR.
- Quién puede fusionar: solo después de alcanzar el número de aprobaciones requerido; el fusor puede ser el creador de la PR, un revisor o un administrador, siempre que la regla se cumpla.

Requerir verificación de estado

- Quién habilita la política: Administrador (o usuario con permiso Maintain) selecciona los checks (CI, pruebas, análisis de seguridad, etc.) que deben pasar.
- Quién ejecuta los checks: el sistema de integración continua u otras herramientas automáticas.
- Quién intenta fusionar: cualquier usuario con permiso Write o superior, pero la fusión queda bloqueada hasta que todos los checks indiquen “success”.

Excepción: si la organización permite “Allow administrators to skip checks”, el administrador puede sobrescribir la regla.

Requerir que las ramas estén actualizadas

- Quién activa la regla: Administrador (o quien tenga permiso Maintain).
- Quién debe actualizar la rama de características: los revisores y, en su caso, el propio autor de la PR. Deben hacer rebase o merge de la rama de destino (por ejemplo, main) y volver a subir los cambios.
- Quién puede fusionar: cualquiera con permiso Write o superior que haya confirmado que la rama está al día y que se cumplan las demás políticas.

Permitir la fusión de confirmaciones (squash merging)

- Quién permite esta opción: Administrador (o usuario con permiso Maintain) al activar “Allow squash merging”.

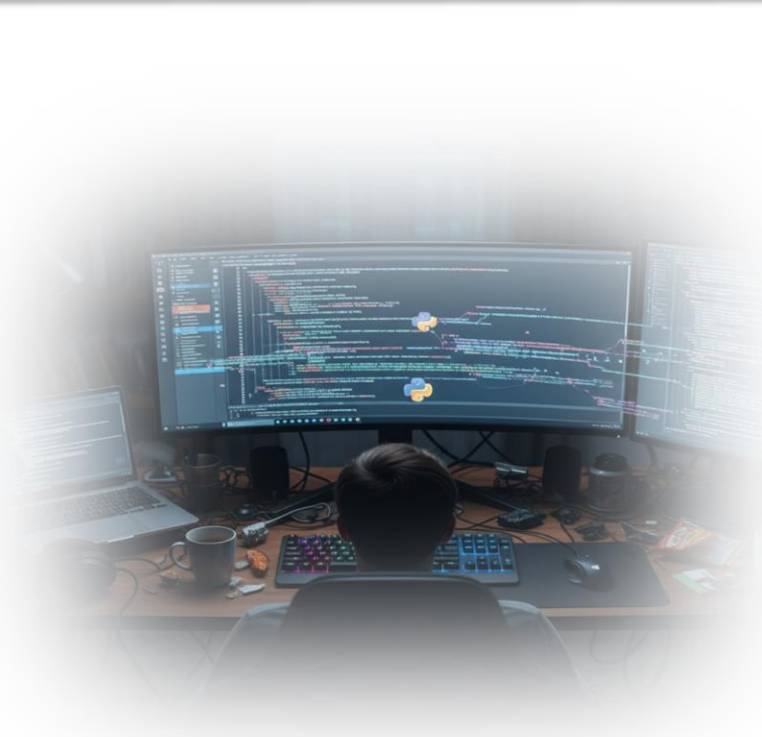
- Quién realiza la fusión con squash: cualquier usuario con permiso Write o superior que tenga la autorización para fusionar (es decir, que haya superado las revisiones, los checks y la actualización de rama).

POLÍTICAS DE PETICIONES PULL REQUESTS.

- Requerir una solicitud de extracción antes de fusionar
 - Quién establece la regla: Administrador (o usuario con permiso Maintain).
 - Quién la cumple: cualquier colaborador que quiera integrar cambios debe crear una pull-request; sin ella no se permite la fusión.
- Exigir que se aprueben las comprobaciones de estado antes de la fusión.
 - Quién habilita la política: Administrador (o usuario con permiso Maintain) define los checks obligatorios (CI, pruebas, análisis de seguridad, etc.).
 - Quién ejecuta los checks: el sistema de integración continua u otras herramientas automáticas.
 - Quién intenta fusionar: cualquier usuario con permiso Write o superior, pero la fusión queda bloqueada hasta que los checks muestren “success”.
 - Posible excepción: si la organización permite que los administradores omitan los checks, ellos pueden hacerlo.
- Se quiere 2 aprobaciones para la extracción
 - Quién configura cuántas aprobaciones son necesarias: Administrador (o quien tenga permiso Maintain).
 - Quién debe aprobar: los revisores designados, que deben poseer permiso Write o superior y marcar “Approve”.
 - Quién puede fusionar: cualquier usuario con permiso Write o superior que haya obtenido las 2 aprobaciones requeridas.
- Es necesario una revisión de mínimo 2 integrantes del equipo
 - Quién define el número mínimo de revisores: Administrador (o usuario con permiso Maintain).

- Quién realiza la revisión: al menos dos miembros del equipo con permiso Write o superior. Cada uno debe aprobar la PR para que la fusión sea posible.
- Se requiere una documentación hacia el apartado ya realizado y de tal manera una revisión y sus respectivas aprobaciones
 - Quién especifica la necesidad de documentación: Administrador (o quien tenga permiso Maintain) activa la política o agrega una “checklist” obligatoria en la plantilla de PR.
 - Quién debe aportar la documentación: el autor de la PR, que incluye el enlace o la descripción del apartado afectado.
 - Quién revisa y aprueba: los revisores asignados (permiso Write o superior) verifican la documentación y otorgan la aprobación.
- Se deberá aprobar que el código sea de buena calidad
 - Quién define la regla de calidad: Administrador (o usuario con permiso Maintain) configura los checks de análisis estático, linters o métricas de cobertura.
 - Quién ejecuta los análisis: herramientas automáticas de CI.
 - Quién valida la calidad: los revisores, que revisan los resultados de los checks y pueden requerir correcciones antes de aprobar.
 - Quién puede fusionar: cualquiera con permiso Write o superior una vez que los checks de calidad hayan pasado y la PR tenga las aprobaciones necesarias.
- Se tendrá que proteger la rama principal del repositorio
 - Quién puede hacer merges a la rama protegida: solo usuarios con permiso Write o superior que cumplan todas las políticas activas, o un Administrador.

CONTROL DE VERSIONES (VERSIONADOR)



Propósito

Establecer normas sobre como se van a ir haciendo los commits y/o versiones internas del sistema web, en esta parte se establecerá; herramientas a utilizar para un correcto formateo de código, convenciones, buenas practicas. Asimismo se establece este presente documento, por el uso de kanban, ya que requiere una buena coordinación y una entrega continua. Mediante lo establecido, se evitara repetición de código o choques de commits. Finalmente, mediante este documento se asegura que los futuros desarrolladores sepan que reglas de pull requests del dicho proyecto.

Alcance

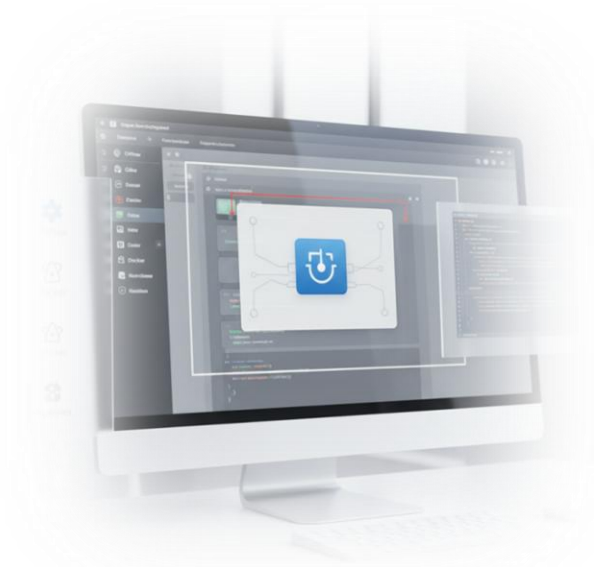
A continuación se establecerán los puntos que se revisaran y/o controlaran en cada pull requests y/o commits:



- 1) Revisar que los issues cumplan las buenas practicas establecidas en el presente documento.
- 2) Revisar que el formateo configurado de código no se cambie.
- 3) Revisar que cada commit siga las buenas practicas establecidas en el presente documento.
- 4) Revisar que se cumpla la función de cada rama que esta establecida.
- 5) Revisar que se cumpla el patrón de carpetas y/o directorio ya establecido.

Finalmente, que se cumpla todo el flujo de trabajo establecido.

CONFIGURACIÓN INICIAL





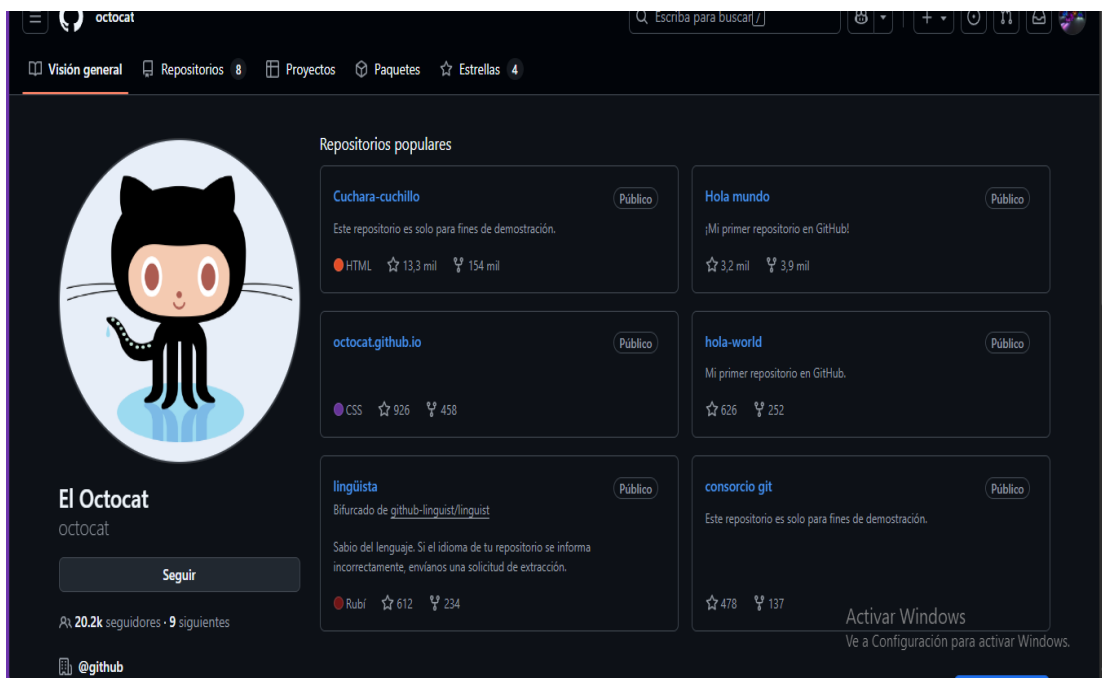
Para la gestión de versiones se ocupara Github y Github Desktop.

CREACIÓN Y CONFIGURACIÓN DE CUENTA GITHUB

Para comenzar con GitHub, deberá crear una cuenta personal gratuita y verificar su dirección de correo electrónico.

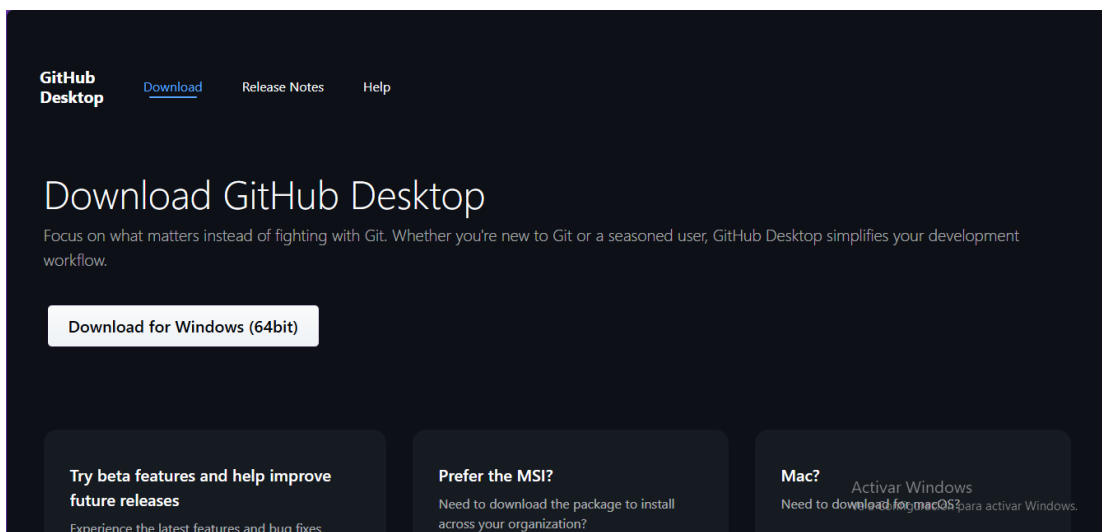
También puede autenticarse con Google o Apple, que son los proveedores de inicio de sesión social admitidos cuando crea su cuenta en GitHub. Para los usuarios de iOS, incluso si ha habilitado la configuración "Ocultar mis direcciones de correo electrónico" para su cuenta de Apple, el uso del inicio de sesión social dará como resultado la creación de una nueva cuenta de GitHub.

Cada persona que usa GitHub inicia sesión en una cuenta de usuario. Tu cuenta de usuario es tu identidad en GitHub y tiene un nombre de usuario y un perfil. Por ejemplo:



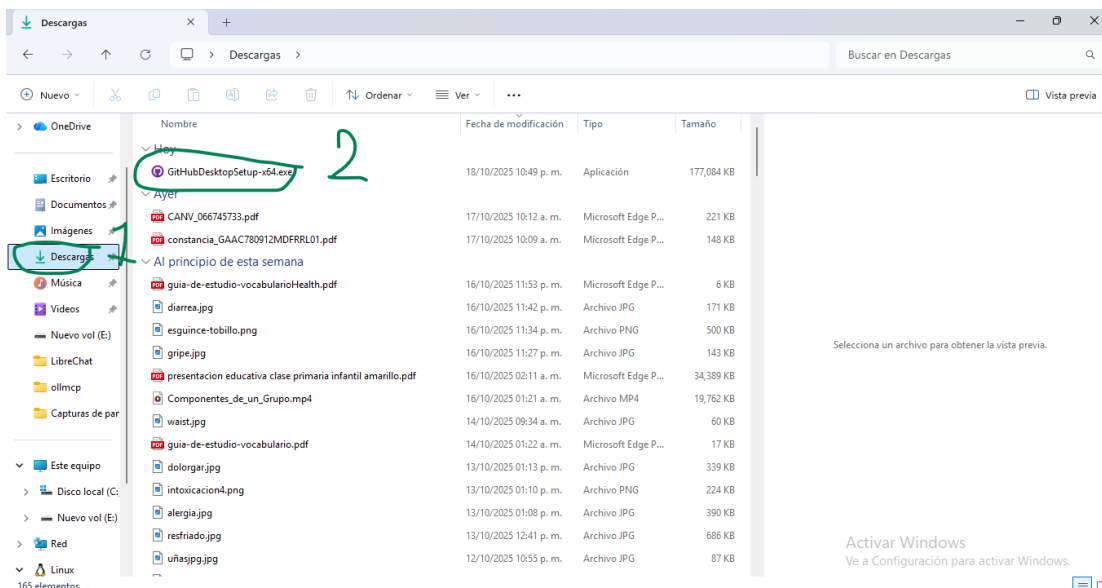
Instalación de Github Desktop

1. Entrar a la página y descargar el instalador: <https://desktop.github.com/download/>



Se debe dar click en “Download For Windows (64bit)”.

2. En tu computadora, dirigirte a la carpeta de descargas y hacer doble clic en el instalador.



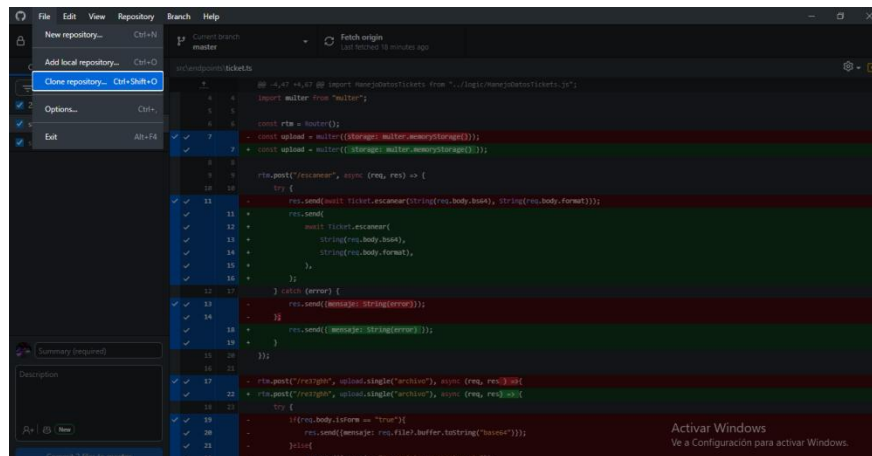


- Después inicia Github Desktop y inicia sesión con tu cuenta de Github.

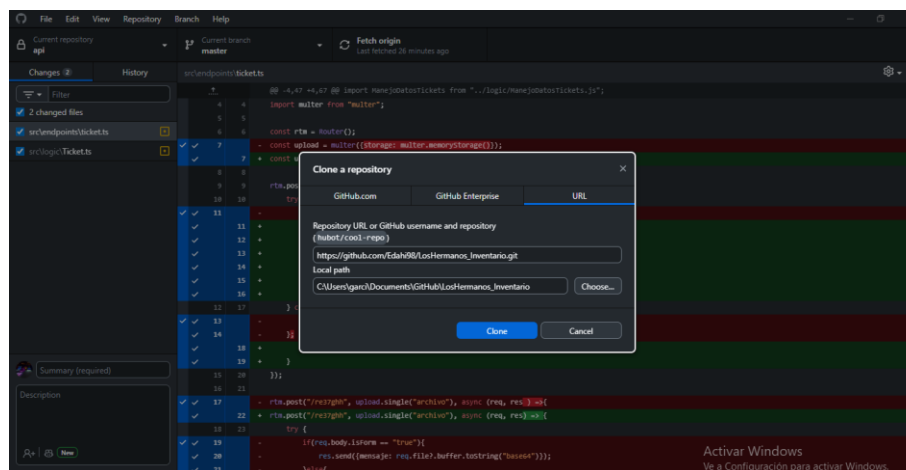
Clonar repositorio

- Entrar a github desktop

Haz clic en “File” y después en “Clone Repository”:

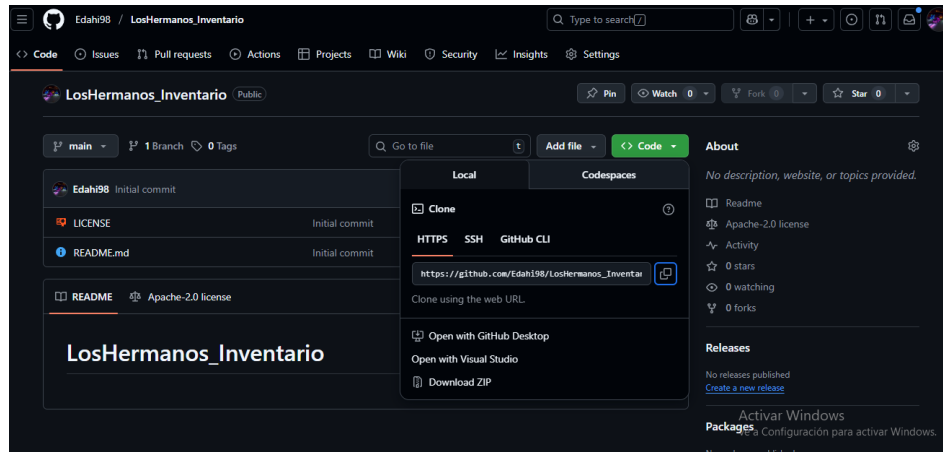


- Dirigirte a la parte de URL, después ingresas la url del repositorio y elige la ruta donde se va a clonar el repositorio. Finalmente dar clic en “Clone”.





Para obtener la url, entra a github en la web y dirigirte al repositorio, y haz clic en “Code” y después copia la url del apartado https



Crear rama

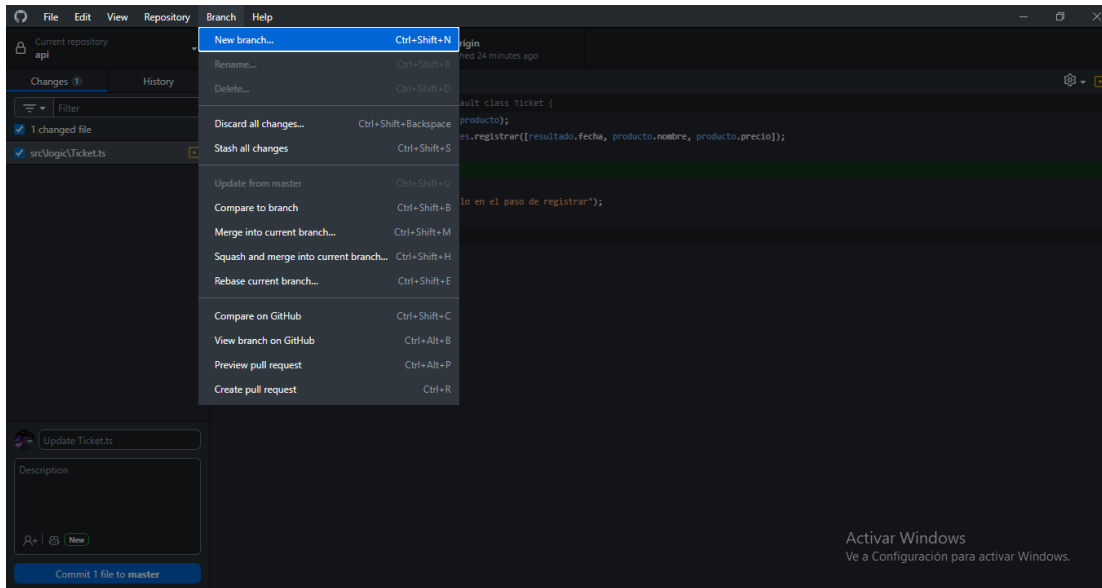
Existen dos formas de crear una rama.

En Desktop:

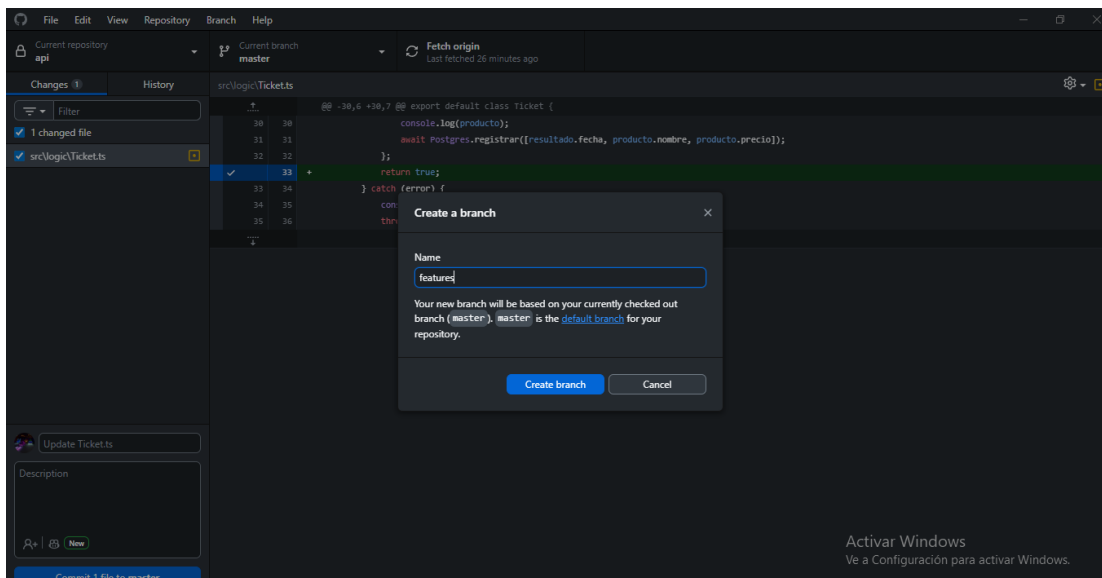
6. Se debe hacer clic en “Branch” y luego en “New Branch”



Reparaciones y venta de autopartes

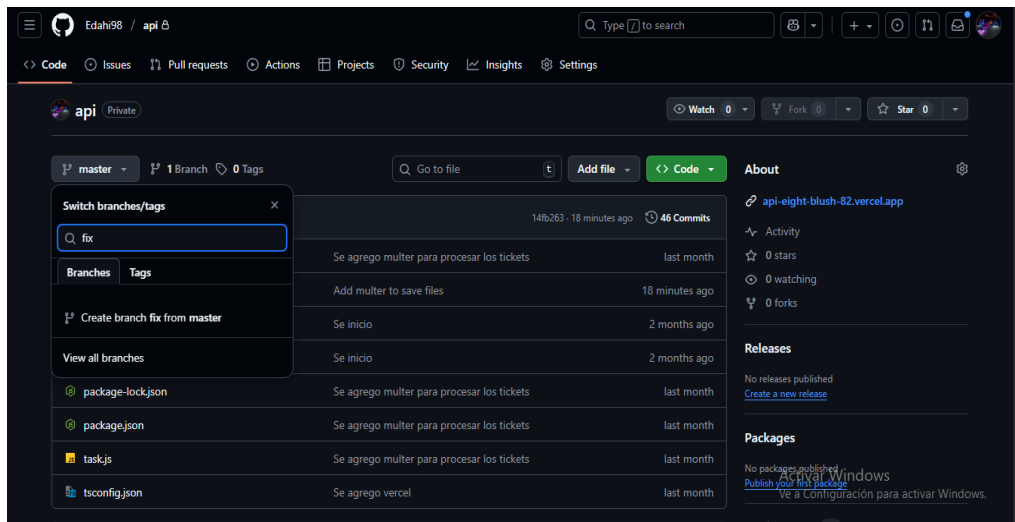


7. Después se ingresa el nombre y se da clic en “Create”.

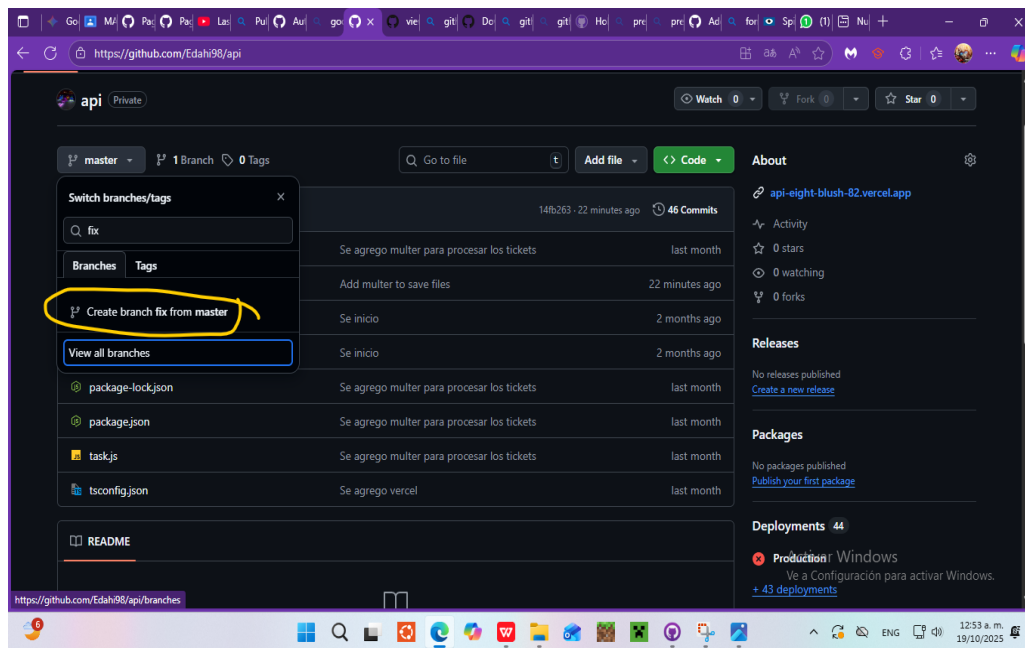


En Web:

8. Se da clic en la rama actual, despues ingresa su nombre en “Find or create a branch...”.



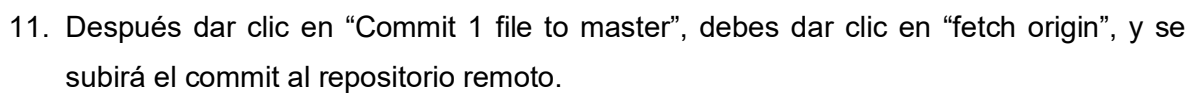
9. Se da clic en “Create branch “ y creara la nueva rama.

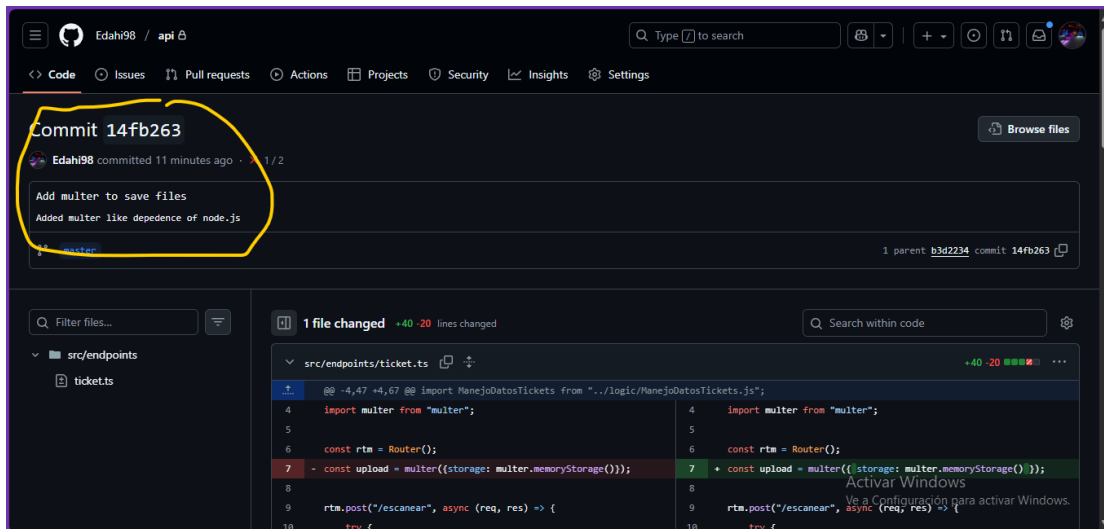
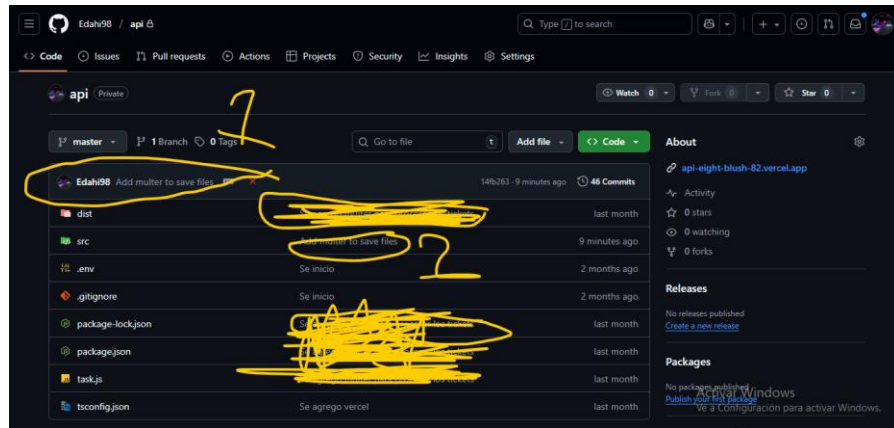


Subir commit

10. Describir commit, elegir la rama y dar commit. Siga la numeración de los pasos.

Nota: En la parte 1, se deben seleccionar los archivos mediante el checkbox.





ESTRUCTURA DEL REPOSITORIO

Estructura de directorios y archivos en laravel:

Directorios principales:



Reparaciones y venta de autopartes

mi-proyecto-laravel/

- | app/ # Lógica principal de la aplicación
- | bootstrap/ # Archivos de inicialización
- | config/ # Configuraciones de la aplicación
- | database/ # Migraciones, seeds y factories
- | public/ # Punto de entrada y assets públicos
- | resources/ # Vistas, assets sin compilar
- | routes/ # Definición de rutas
- | storage/ # Archivos generados por el framework
- | tests/ # Pruebas automatizadas
- | vendor/ # Dependencias de Composer
- | varios archivos de configuración

Copy

Detalle de cada directorio:

app/ - Lógica de la aplicación

app/

- | Console/ # Comandos Artisan personalizados
- | Exceptions/ # Manejo de excepciones
- | Http/
 - | Controllers/ # Controladores
 - | Middleware/ # Middleware personalizados
 - | Requests/ # Form requests (validación)
- | Models/ # Modelos Eloquent
- | Providers/ # Service providers

Copy

bootstrap/ - Inicialización

bootstrap/

- | app.php # Punto de arranque del framework
- | cache/ # Archivos de caché del framework

Copy



config/ - Configuraciones

Copy

```
config/
├─ app.php           # Configuración general
├─ auth.php          # Configuración de autenticación
├─ cache.php         # Configuración de caché
├─ database.php      # Configuración de base de datos
├─ filesystems.php   # Sistema de archivos
├─ mail.php          # Configuración de email
├─ queue.php         # Configuración de colas
├─ services.php      # Configuración de servicios
└─ ... otros archivos
```

database/ - Base de datos

Copy

```
database/
├─ factories/        # Factories para testing
├─ migrations/       # Migraciones de base de datos
├─ seeders/          # Seeders para poblar BD
└─ (puede contener SQLite database)
```

public/ - Punto de entrada público

Copy

```
public/
├─ index.php         # Punto de entrada de todas las requests
├─ .htaccess         # Configuración Apache
├─ favicon.ico       # Favicon
└─ assets/           # CSS, JS, imágenes compilados
```



resources/ - Recursos sin compilar

```
resources/  
├─ css/           # Archivos CSS fuente  
├─ js/            # Archivos JavaScript fuente  
├─ views/         # Vistas Blade  
│   └─ welcome.blade.php # Vista de bienvenida por defecto  
└─ lang/          # Archivos de idiomas
```

[Copy](#)

resources/ - Recursos sin compilar

```
resources/  
├─ css/           # Archivos CSS fuente  
├─ js/            # Archivos JavaScript fuente  
├─ views/         # Vistas Blade  
│   └─ welcome.blade.php # Vista de bienvenida por defecto  
└─ lang/          # Archivos de idiomas
```

[Copy](#)

routes/ - Definición de rutas

```
routes/  
├─ web.php        # Rutas web (con sesión, CSRF)  
├─ api.php        # Rutas API (stateless)  
├─ console.php    # Comandos de consola  
└─ channels.php   # Canales de broadcasting
```

[Copy](#)

storage/ - Archivos generados

```
storage/  
├─ app/           # Archivos de la aplicación  
│   └─ public/    # Archivos públicos (storage:link)  
├─ framework/     # Archivos del framework  
│   └─ cache/     # Caché  
│   └─ sessions/  # Sesiones  
│   └─ views/     # Vistas compiladas  
└─ logs/          # Archivos de log
```

[Copy](#)



tests/ - Pruebas

Copy

```
tests/  
├─ Feature/      # Pruebas de características  
├─ Unit/         # Pruebas unitarias  
└─ TestCase.php  # Clase base para pruebas
```

Estructura de directorios y archivos en el micro-servicio:

Copy

```
inventory-microservice/  
├─ src/  
│   └─ features/  
│       └─ inventory/      # Gestión de inventario  
│           ├── inventory.controller.js  
│           ├── inventory.service.js  
│           ├── inventory.routes.js  
│           ├── inventory.model.js  
│           ├── inventory.validators.js  
│           └─ inventory.builder.js # Patrón Builder para productos  
│       └─ taxes/          # Gestión de impuestos  
│           ├── tax.controller.js  
│           ├── tax.service.js  
│           ├── tax.routes.js  
│           ├── tax.model.js  
│           └─ tax.validators.js
```



Reparaciones y venta de autopartes

```
| | | └─ tax.builder.js      # Patrón Builder para cálculos fiscales
| | | └─ transactions/      # Transacciones de stock
| | |   └─ transaction.controller.js
| | |   └─ transaction.service.js
| | |   └─ transaction.routes.js
| | |   └─ transaction.model.js
| | |   └─ transaction.validators.js
| | |   └─ transaction.builder.js # Builder para transacciones
| | └─ core/
| |   └─ database/          # Configuración Sequelize
| |     └─ config.js
| |     └─ connection.js
| |     └─ migrations/      # Migraciones de PostgreSQL
| |   └─ builders/          # Builders compartidos
| |     └─ base.builder.js
| |     └─ price.builder.js  # Builder especializado en precios
| |     └─ validators/      # Validaciones compartidas
| └─ shared/
|   └─ utils/
|     └─ logger.js
|     └─ math.js            # Cálculos matemáticos precisos
|     └─ formatters.js      # Formateo de precios
|   └─ constants/

| | | └─ tax-types.js      # Tipos de impuestos (IVA, ISR, etc.)
| | | └─ inventory-status.js # Estados del inventario
| | └─ errors/
| |   └─ inventory-errors.js
| |   └─ tax-errors.js
| |   └─ middleware/
| |     └─ error-handler.js
| |     └─ validation.js
| |     └─ logging.js
| └─ app.js
└─ tests/
  └─ features/
    └─ inventory/
    └─ taxes/
    └─ transactions/
  └─ integration/
└─ docs/
  └─ api.md
  └─ tax-calculation.md
└─ docker/
└─ .env
```

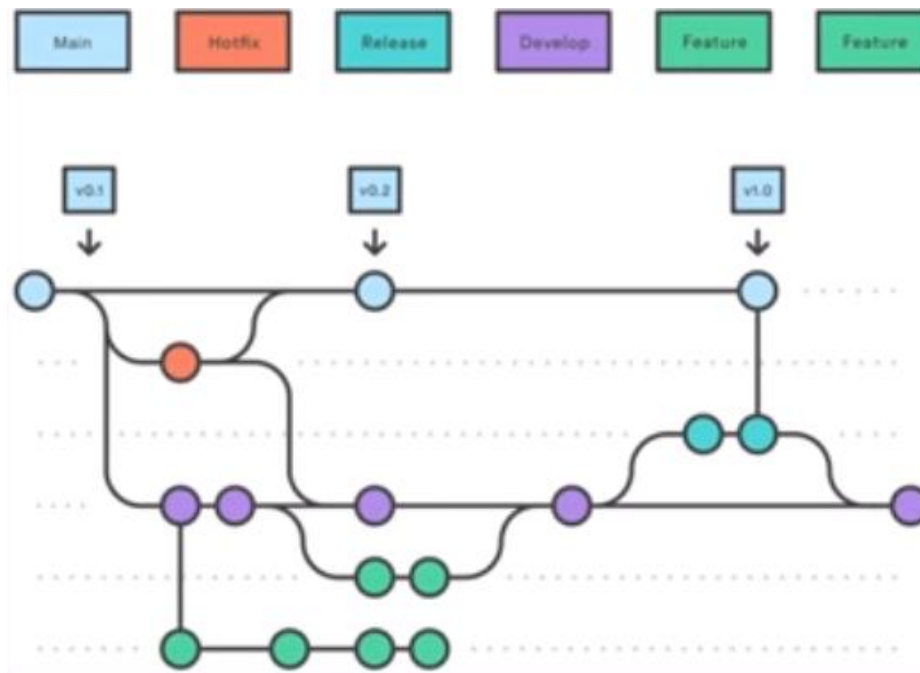
```

├─ .sequelizerc           # Configuración Sequelize
├─ package.json
└─ server.js
  
```

```

├─ .prettierrc           # Configuración Prettier
└─ .prettierignore       # Archivos a ignorar por Prettier
  
```

Ramas principales establecidas en el proyecto:



- Main o master: Está rama es la principal, no se puede alterar, sin antes pasar por develop, feature y release. Sera la rama que se ocupa para desplegar y/o en producción la página web.
- Convención de versión mayor o menor como ejemplo:
 - ◆ 1.0.0 - Duck Update
 - ◆ 2.0.0 - Snake Update
 - ◆ 2.1.0 - Drop Extra of Snake Update.



- Hotfix: Está sirve para corregir errores que se encuentren en la versión desplegada y/o en producción. Por ejemplo; tiene problemas al realizar una compra mediante tarjeta de crédito.
 - Convención de versión hotfix como ejemplo:
 - ◆ 1.1.1
 - ◆ 1.2.10
- Feature: Esta rama es temporal, se va a desarrollar un cambio que aun no este confirmado. Este feature se crea a partir de develop y después se va integrar en develop. Este feature necesita ser aceptado por el equipo y hablado como va a afectar su implementación en develop.
 - Ejemplo:
 - ◆ Se elimino el apartado de inventario.
- Develop: Se integran todas características que están en la rama feature.
 - Ejemplo:
 - ◆ Se agrego la función que permite exportar las calificaciones en archivo del tipo Excel.
 - ◆ Se agrego la posibilidad de integrar Google Calendar.
- Release: Verifica que todas las características y/o requerimientos estén cumplidos de acuerdo a kanban antes de mandarlo a main/master para ponerlo en producción.

Como se deberán nombrar los commits y futuras ramas

Commits

El presente apartado proporciona un conjunto sencillo de reglas para crear un historial de commits explícito; lo que facilita la escritura de herramientas automatizadas encima. Esta

convención encaja con SemVer, describiendo las características, las correcciones y los cambios importantes realizados en los mensajes de los commits.

Cada commit debe cumplir con la siguiente estructura:

<tipo>[módulo]: descripción

[pie] / breaking change: <texto>

- **Tipos** que estarán presentes inicialmente.
 - **fix**: Indica que parchaste un bug en el código.
 - **feat**: Indica que se introdujo una nueva característica.
 - **revert**: Indica que se eliminó una característica.
 - **edit-feat**: Indica que se editó una característica.
 - **refactor**: Indica que se alteró alguna función.
- ◆ **Reglas** que deben de seguir los tipos futuros.
 - 1) Pueden estar en UPPERCASE, CamelCase, kebab-case, PascalCase, Sentence case, snake_case y Start Case.
 - 2) Nunca debe estar vacío.
 - 3) Nunca debe terminar en punto.
 - 4) No tiene límite de tamaño.
 - 5) Pueden comenzar con signo de exclamación.
 - 6) Se recomiendan que los futuros tipos sean; build, chore, ci, docs, perf, style, test.
- **Pie**.
- **Módulo**: Es opcional, en se pone el módulo, por ejemplo; crud, etc.

- Este puede indicar cambios importantes, que serían los; BREAKING CHANGES:
 - ◆ Importancia: En el mundo de la gestión de software existe un lugar temido llamado "Infierno de dependencia". Cuanto más grande crezca su sistema y más paquetes integrado en su software, más probabilidades tendrá de encontrarse a sí mismo, día, en este pozo de desesperación. En sistemas con muchas dependencias, el lanzamiento de nuevas versiones de paquetes puede convertirse en una pesadilla. Si las especificaciones de dependencia son demasiado estrictas, está en peligro de bloqueo de versión (la incapacidad de actualizar un paquete sin tener que liberar nuevas versiones de cada paquete dependiente).
 - ◆ Dado el número de versión, es importante subir el grado de importancia. A continuación se enlistan los que se deben usar para los commits:
 - I. MAJOR: La versión MAJOR cuando realizas un cambio incompatible en el API.
 - II. MINOR: La versión MENOR cuando añades funcionalidad compatible con versiones anteriores.
 - III. PATCH: La versión PARCHE cuando reparas errores compatibles con versiones anteriores.

Ejemplos:

A. Mensaje de commit con descripción y con BREAKING CHANGE:

feat: allow provided config object to extend other configs



BREAKING CHANGE: `extends` key in config file is now used for extending other config files

- B. Mensaje de commit con ! para llamar la atención para el BREAKING CHANGE:

feat!: send an email to the customer when a product is shipped.

BREAKING CHANGE: `extends` key in config file is now used for extending other config files

- C. Mensaje de commit con el módulo y !, además con el BREAKING CHANGE.

feat(api)!: send an email to the customer when a product is shipped

BREAKING CHANGE: `extends` key in config file is now used for extending other config files.

Ramas

CONVENCIONES EN LA NOMENCLATURA PARA RAMAS

- Para las ramas, se usara la estructura <token>/<short-descriptive-name>.
- Para los nombres que siguen al token, buscar palabras cortas que mejor describan en lo que se está trabajando. Ejemplo: routing, profile-settings, course-management.
- Evita caracteres especiales o espacios.
- Sus nombres deben transmitir mucha información sobre qué cambios se incluyen en la rama, quién está trabajando en ella y en qué etapa del proceso se encuentra.
- En algunos casos, podria ser útil incluir el nombre de usuario del desarrollador en el nombre de la rama. Esto puede facilitar ver quién está trabajando en qué. Por ejemplo, si tu nombre de usuario es jsmith y estás trabajando en una corrección de error, podrías nombrar tu rama jsmith/hotfix.



- Se debe usar letras minúsculas para los nombres de las ramas para evitar confusiones, ya que Github distingue entre mayúsculas y minúsculas.
- No se deben ocupar nombres de ramas largos y complejos.

Ejemplos:

- chore/deps-upgrade
- docs/setup
- feature/project-roadmap
- fix/student-routing
- refactor/settings-components
- test/cohort
- edahi/feature/setup

FLUJO DE TRABAJO

Creación de ramas

- A. Escoger un nombre descriptivo y con prefijo
 - a) En GitHub, desde la página del repositorio, pulsa Branch: main -> New branch, escribe el mismo prefijo.
 - b) Los prefijos (feature/, hotfix/) facilitan la identificación del tipo de trabajo y reducen confusión en CI/CD,
- B. Crear la rama y cambiar a ella
 - a) La rama recién creada queda seleccionada automáticamente en la UI.
 - b) Mantiene el trabajo aislado del main.

Trabajo en la rama – commits

- a) Commits atómicos



- a) Cada commit contiene una única unidad lógica (p. ej. “añadir validación de email”).
- b) Facilita volver a un punto concreto si se necesita deshacer.
- b) Mensajes descriptivos
 - a) Usa el formato Conventional Commits (type(scope): short description). Ej.: feat(auth): add JWT login.
 - b) Hace que el historial sea legible y permite generar changelogs automáticamente.
- c) Frecuencia
 - a) Haz push después de cada commit relevante (al menos una vez al día).
 - b) Mantiene el trabajo actualizado en el servidor y permite que los revisores vean cambios temprano.
- d) Pruebas locales
 - a) Ejecuta la suite de tests antes de push.
 - b) Reduce la probabilidad de que la CI falle.

Pull Request (PR) – abrir, revisar y aprobar

- a. Abrir PR
 - a) Desde la rama recién creada, pulsa Pull requests → New pull request, elige base: main y compare: feature/login-ui. Se escribe un título descriptivo y una descripción que referencia el issue (Closes #123).
 - b) La PR queda visible para todo el equipo.
- b. Asignar revisores
 - a) Usa la sección Reviewers para añadir 1-3 personas (o se asignan automáticamente por CODEOWNERS).
 - b) Garantiza que alguien con conocimiento del área revise el cambio.



- c. Revisión
 - a) Cada revisor inspecciona los archivos, comenta inline, y pulsa Request changes o Approve.
 - b) Los comentarios obligan al autor a corregir problemas antes del merge.
- d. CI automática
 - a) Al crear o actualizar la PR, se disparan los pipelines. Los status checks aparecen bajo la PR. El merge está bloqueado mientras alguno falle.
 - b) La política de rama protegida requiere que todos los checks pasen antes de permitir el merge.
- e. Aprobación final
 - a) Cuando al menos el número requerido de revisores aprueba y los checks son verdes, el botón Merge pull request se habilita.
 - b) Se asegura que el código ha pasado revisión humana y automática.

Fusión y eliminación de ramas

- a) Merge (squash)
 - a) En la PR, pulsa Squash and merge (o Rebase and merge si prefieres historial lineal).
 - b) Mantiene la historia limpia; cada PR se representa como un solo commit.
- b) Eliminar rama
 - a) Después del merge, GitHub muestra un botón Delete branch; basta con pulsarlo.
 - b) Libera espacio de nombres y evita confusiones en el futuro.
- c) Protección contra borrado accidental
 - a) En Settings -> Branches, marca Prevent deletion y Disallow force pushes.



- b) Evita que se elimine o sobrescriba la rama principal por accidente.

REVISIÓN Y APROBACIÓN DEL CÓDIGO

1. Creación de la PR
 - a) El autor abre la PR desde una rama corta y descriptiva y escribe una descripción que referencia el issue correspondiente (Closes #123).
 - b) El equipo sabe qué y por qué se está proponiendo el cambio.
2. Asignación de revisores
 - a) Con “Reviewers” se añaden 1-3 personas (o, si existe CODEOWNERS, se asignan automáticamente).
 - b) Garantiza que al menos un experto revise la zona afectada del código.
3. Primer vistazo (botón “Files changed”)
 - a) Cada revisor inspecciona los archivos modificados.
 - b) Detecta errores visibles, problemas de estilo o arquitectura antes de ejecutar la CI.
4. Comentarios y “Request changes”
 - a) Si algo no está bien, el revisor usa la herramienta de review para dejar comentarios inline o cambiar el estado a Request changes.
 - b) Obliga al autor a corregir los problemas antes de que la PR pueda ser aprobada.
5. Correcciones del autor
 - a) El autor hace los cambios y los vuelve a someter a revisión.
 - b) Cierra el ciclo de feedback y mantiene la PR actualizada.
6. Aprobación
 - a) Cuando el revisor está satisfecho pulsa Approve.



- b) Marca que el cambio cumple con los criterios y que el autor ha atendido los comentarios.
- 7. Cumplimiento de la política de protección de rama
 - a) La rama objetivo (ej. main) suele estar protegida y requiere al menos N aprobaciones y que todos los checks de CI pasen antes de permitir el merge.
- 8. Merge
 - a) Con Squash-and-merge o Rebase-and-merge el código se integra en la rama principal manteniendo un historial limpio.
 - b) El historial queda lineal y cada PR aparece como un único commit atómico.

Criterios de revisión

- a) Funcionalidad
 - a) ¿El cambio implementa lo solicitado en el issue? ¿Se cubren los casos límite?
 - b) Revisa pruebas manuales o pasos de verificación del issue.
- b) Pruebas automatizadas
 - a) ¿Hay pruebas unitarias / de integración que cubran la nueva lógica? ¿La cobertura sigue por encima del umbral (p.e. 80 %)?
 - b) La CI ejecutará la suite; la ausencia de pruebas hará fallar el check.
- c) Calidad de código
 - a) ¿El código sigue las guías de estilo?. ¿Los nombres son claros y descriptivos?
 - b) Linters se pueden ejecutar como parte de CI.
- d) Seguridad
 - a) Se han revisado dependencias vulnerables? ¿No se exponen datos sensibles?
 - b) Escáneres de seguridad en CI detectan problemas automáticamente.



- e) Rendimiento
 - a) ¿El cambio afecta tiempos de respuesta o consumo de recursos? ¿Se incluyeron benchmarks si procede?
 - b) Opcional, pero recomendable para áreas críticas.
- f) Documentación
 - a) ¿Se actualizó el README, los diagramas o los comentarios de API? ¿El changelog incluye la entrada correspondiente?
 - b) Sin doc, la PR puede ser rechazada.
- g) Impacto en arquitectura
 - a) ¿Se introducen dependencias nuevas? ¿Se rompe la modularidad?
 - b) Revisa diagramas.
- h) Mantenimiento
 - a) ¿El código es fácil de leer y de probar en el futuro? ¿Se sigue el patrón de commits “Conventional Commits”?
 - b) Mejora la trazabilidad y la generación automática de changelog.

Automatización del control de estilo con Prettier en Pull Requests

- i) Disparador del workflow
 - a) Descarga el código de la PR.
 - b) Archivo disponible para el siguiente paso.
- j) Fases que la pipeline ejecuta (en orden)
 - a) Ejecuta npm install para que Prettier esté disponible.



- b) Todas las dependencias incluidas.
- k) Check de estilo
 - a) Ejecuta `npx prettier --check ..` Prettier inspecciona todos los archivos del proyecto y devuelve un código de salida 0 solo si cada archivo está ya formateado.
 - b) Si algún archivo no cumple, el job termina con failure y la PR muestra el mensaje "Prettier check failed".
- l) (Opcional) Comentario automático
 - a) GitHub Actions bot publica en la PR una lista de los archivos que no pasan el check y sugiere ejecutar `npx prettier --write .` localmente.
 - b) El autor recibe feedback inmediato sin necesidad de buscar el log.
- m) (Opcional) Auto-fix
 - a) Si la política del proyecto lo permite, el mismo workflow puede generar un commit en la rama de la PR con los cambios de formato (`prettier --write + git commit -am "style: apply Prettier"`). El commit se crea bajo la cuenta del bot y la PR vuelve a quedar "green".
 - b) El merge se desbloquea sin que el autor tenga que volver a hacer push.
- n) Publicación del status check
 - a) GitHub marca la verificación como passed o failed según el resultado de la fase C (o E, si se usa auto-fix).
 - b) La regla de protección de rama permite el merge solo cuando el check está passed.

RESOLUCIÓN DE CONFLICTOS

Detección de conflictos

- Lista de Pull Requests -> la tarjeta del PR muestra el banner rojo "This branch has conflicts that must be resolved before merging".



- Pestaña “Files changed” -> aparecen archivos marcados con “conflicting files”.

Comprender el conflicto

- Entra al PR y selecciona la pestaña “Files changed”.
- Haz clic en “Resolve conflicts” (solo disponible si el conflicto es simple). GitHub te muestra un editor con los marcadores y el código de ambas ramas.

Resolución de conflictos

1. Edita el archivo en el cuadro que aparece: elimina los marcadores y deja la versión final que desees. Decide si mantienes los cambios de la rama base, los de tu rama, o una combinación de ambos].
2. Cuando termines, pulsa “Mark as resolved”. Repite el procedimiento para cada archivo marcado como conflictivo.
3. Finalmente, pulsa “Commit merge” (o “Commit changes” según la versión de la UI). Con esto GitHub crea automáticamente un commit en la rama head con la resolución del conflicto.
4. Conflictos complejos – crear/editar en la web.
 - a) En la página del PR, bajo la barra de Conversation, haz clic en “Edit file” (puedes crear un nuevo archivo o editar el existente) y selecciona “Create a new branch for this commit and start a pull request”.
 - b) Realiza los cambios necesarios en el editor, elimina los marcadores y guarda el archivo (commit directo a la rama del PR).
 - c) Repite para cada archivo en conflicto. Cuando todos estén sin marcadores, el PR volverá a mostrarse como “Ready to merge”.

Mantener la rama sincronizada (opcional, pero recomendado)



Antes de intentar resolver, verifica si puedes actualizar la rama con la base usando el botón “Update branch”:

5. El botón aparece cuando la rama head está desactualizada pero no tiene conflictos. Al pulsarlo GitHub hace un merge (o rebase si eliges la opción del menú) de la rama base en tu PR, manteniendo el historial limpio.
6. Si el botón está deshabilitado, significa que hay conflictos que deben resolverse primero (vuelve a los pasos 3-a o 3-b).

Post-resolución: merge seguro

- CI vuelve a ejecutarse automáticamente al crear el commit de resolución. Asegúrate de que todos los checks pasan.
- Revisión final: al menos un revisor debe aprobar el PR (las reglas de protección de rama pueden exigir N aprobaciones).
- Merge: usa “Merge pull request” (squash-or-merge o rebase-and-merge según la política del proyecto).
- Borrar la rama: después del merge, GitHub mostrará la opción “Delete branch” – haz clic para mantener el repositorio limpio.

MANEJO DE VERSIONES Y REALESSES

Especificación del Versionado Semántico.

Las palabras clave “DEBE”, “NO DEBE”, “OBLIGATORIO”, “DEBERÁ”, “NO DEBERÁ”, “DEBERÍA”, “NO DEBERÍA”, “RECOMENDADO”, “PUEDE” y “OPCIONAL” en este documento serán interpretadas como se describe en [RFC 2119-es](#).



- IV. Un número de versión normal DEBE tener la forma de X.Y.Z donde X, Y y Z son números enteros no negativos, y NO DEBEN ser precedidos de ceros. X es la versión mayor, Y es la versión menor, y Z es la versión parche. Cada elemento DEBE incrementarse numéricamente. Por ejemplo: 1.9.0 -> 1.10.0 -> 1.11.0
- V. Una vez que el paquete versionado ha sido publicado, el contenido de esa versión NO DEBE ser modificado. Cualquier modificación DEBE ser publicada como una nueva versión.
- VI. Una versión mayor en cero (0.y.z) se considera como desarrollo inicial. Todo PUEDE cambiar en cualquier momento.
- VII. La versión parche Z (x.y.Z $x > 0$) DEBE ser incrementada si solamente se introducen correcciones de errores compatibles con versiones anteriores. Una corrección de error se define como un cambio interno que corrige un comportamiento incorrecto.
- VIII. La versión menor Y (x.Y.zx $x > 0$) DEBE ser incrementada si se introduce funcionalidad nueva y compatible con la versión anterior del API público. Ésta DEBE ser incrementada si se introduce cualquier funcionalidad al API público o mejora al código privado. Este PUEDE incluir cambios a nivel de parches. La versión parche DEBE reiniciarse a 0 cuando una versión menor se incrementa.
- IX. La versión mayor (X.y.z $X > 0$) DEBE ser incrementada solamente si se introducen cambios incompatibles con la versión anterior del API público. Este PUEDE incluir cambios de nivel menor y parches. Versiones parche y menores DEBEN ser reiniciadas a 0 cuando una versión mayor es incrementada.
- X. Una versión de prelanzamiento PUEDE ser denotada agregando un guión y una serie de identificadores separados por puntos, inmediatamente seguida de la versión parche. Los identificadores DEBEN ser compuestos sólo de caracteres alfanuméricos ASCII y guión ([0-9A-Za-z-]). Los identificadores NO DEBEN estar vacíos. Identificadores numéricos NO DEBEN ser precedidos de cero. Versiones de prelanzamiento tienen una precedencia inferior que una versión normal. Una versión de prelanzamiento indica que esa versión no es estable y puede no satisfacer los requerimientos de



compatibilidad destinados como se denota en la versión normal asociada. Por ejemplo: 1.0.0-alpha, 1.0.0-alpha.1, 1.0.0-0.3.7, 1.0.0-x.7.z.92.

- XI. Metadatos de compilación PUEDEN ser denotados agregando el signo más y una serie de identificadores separados por puntos, inmediatamente seguido de la versión parche o prelanzamiento. Los identificadores DEBEN ser compuestos de sólo caracteres alfanuméricos ASCII y guión ([0-9A-Za-z-]). Los identificadores NO DEBEN estar vacíos. Identificadores numéricos NO DEBEN ser precedidos de cero. Los metadatos de compilación DEBEN ser ignorados cuando se determina la precedencia de la versión. Por lo tanto, dos versiones que difieren solamente en los metadatos de compilación tienen la misma precedencia. Por ejemplo: 1.0.0-alpha+001, 1.0.0+20130313144700, 1.0.0-beta+exp.sha.5114f85.
- XII. La precedencia se refiere a cómo las versiones se comparan entre ellas cuando se ordenan.
- a) La precedencia DEBE ser calculada separando los identificadores de la versión en mayor, menor, parche y prelanzamiento (dejando de lado los metadatos de compilación).
 - b) La precedencia es determinada por la primera diferencia al comparar cada uno de los identificadores de izquierda a derecha como se indica: mayor, menor y parche son siempre comparadas numéricamente. Ejemplo: Por ejemplo: 1.0.0 < 2.0.0 < 2.1.0 < 2.1.1.
 - c) Cuando la versión mayor, menor y parche son iguales, la versión de prelanzamiento tiene menor precedencia que la versión normal. Por ejemplo: 1.0.0-alpha < 1.0.0.
 - d) La precedencia para dos versiones de prelanzamiento con la misma versión mayor, menor y parche DEBE ser determinada comparando cada identificador separado por punto, de izquierda a derecha, hasta que una diferencia sea encontrada como se indica:



- i. identificadores compuestos solamente por números son comparados numéricamente
- ii. los identificadores con letras o guiones son comparados léxicamente en orden ASCII.
- iii. Identificadores numéricos siempre tienen menor precedencia que identificadores no numéricos.
- iv. Un conjunto de campos de prelanzamiento más numeroso tiene mayor precedencia que un conjunto menos numeroso, si todos los identificadores anteriores son iguales.
- v. Ejemplos: $1.0.0\text{-alpha} < 1.0.0\text{-alpha}.1 < 1.0.0\text{-alpha}.beta < 1.0.0\text{-beta} < 1.0.0\text{-beta}.2 < 1.0.0\text{-beta}.11 < 1.0.0\text{-rc}.1 < 1.0.0$.

Importancia de seguir esta especificación

Esto no es una idea nueva o revolucionaria. De hecho, probablemente ya tengas algo parecido a esto. El problema es que “parecido” no significa suficientemente bueno. Sin el cumplimiento de alguna especie de especificación formal, los números de versión son esencialmente inútiles para el manejo de dependencias. Dándole un nombre y una definición clara a las ideas anteriores, se hace fácil comunicar tus intenciones a los desarrolladores de la página web.

Forma Gramatical Backus–Naur para Versiones Válidas SemVer

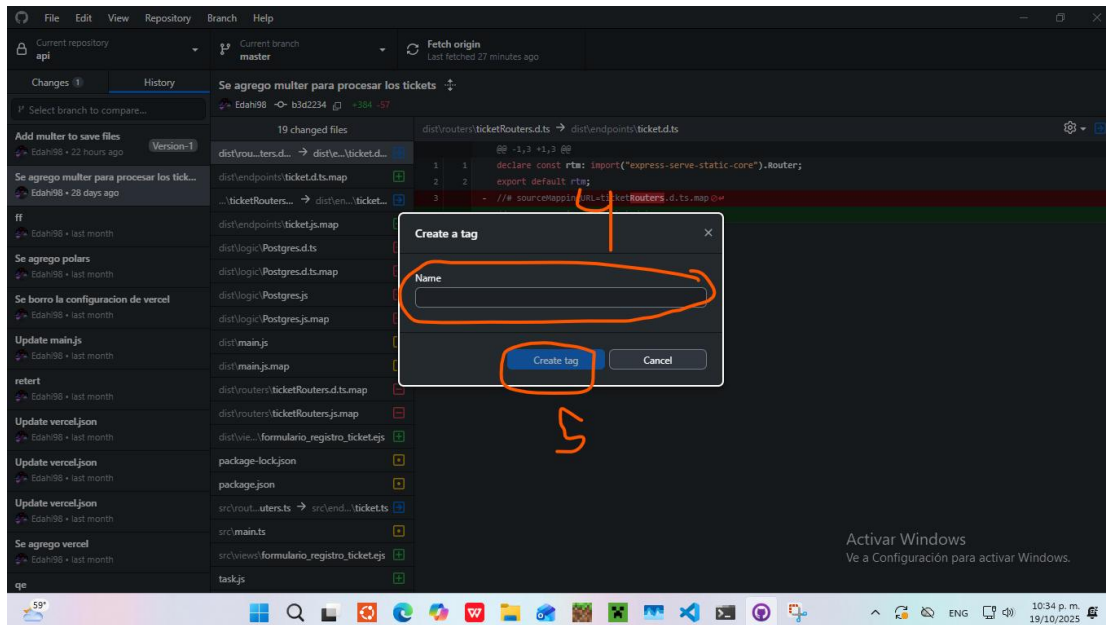
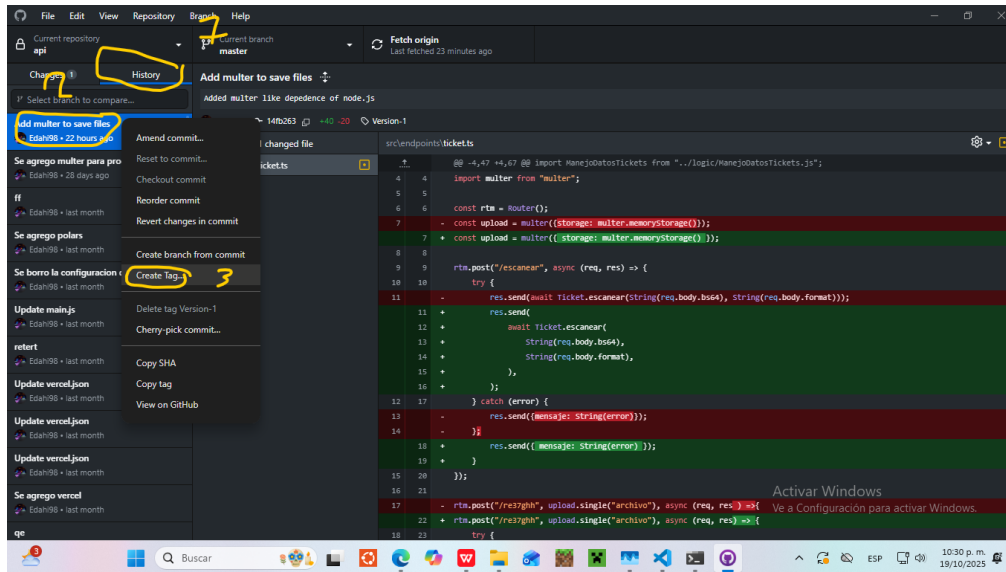
```
1 <semver válido> ::= <versión núcleo>
2 | <versión núcleo> "-" <prelanzamiento>
3 | <versión núcleo> "+" <compilación>
4 | <versión núcleo> "-" <prelanzamiento> "+" <compilación>
5
6 <versión núcleo> ::= <mayor> "." <menor> "." <parche>
7
8 <mayor> ::= <identificador numérico>
9
10 <menor> ::= <identificador numérico>
11
12 <parche> ::= <identificador numérico>
13
14 <prelanzamiento> ::= <identificadores prelanzamiento separados-por-puntos>
15
16 <identificadores prelanzamiento separados-por-puntos> ::= <identificador prelanzamiento>
17 | <identificador prelanzamiento> "." <identificadores prelanzamiento separados-por-puntos>
18
19 <compilación> ::= <identificadores compilación separados-por-puntos>
20
21 <identificadores compilación separados-por-puntos> ::= <identificador compilación>
22 | <identificador compilación> "." <identificadores compilación separados-por-puntos>
23
24 <identificador prelanzamiento> ::= <identificador alfanumérico>
25 | <identificador numérico>
26
27 <identificador compilación> ::= <identificador alfanumérico>
28 | <digitos>
29
30 <identificador alfanumérico> ::= <no-digito>
31 | <no-digito> <caracteres identificadores>
32 | <caracteres identificadores> <no-digito>
33 | <caracteres identificadores> <no-digito> <caracteres identificadores>
34
35 <identificador numérico> ::= "0"
36 | <digito positivo>
37 | <digito positivo> <digitos>
38
39 <caracteres identificadores> ::= <carácter identificador>
40 | <carácter identificador> <caracteres identificadores>
41
42 <identificador carácter> ::= <digito>
43 | <no-digito>
44
45 <no-digito> ::= <letra>
46 | "-"
47
48 <digitos> ::= <digito>
49 | <digito> <digitos>
50
51 <digito> ::= "0"
52 | <digito positivo>
53
54 <digito positivo> ::= "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
55
56 <letra> ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J"
57 | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T"
58 | "U" | "V" | "W" | "X" | "Y" | "Z" | "a" | "b" | "c" | "d"
59 | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" | "n"
60 | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
61 | "y" | "z"
```

PROCESO DE TAGS Y RELEASES

Creación de un tag



Reparaciones y venta de autopartes



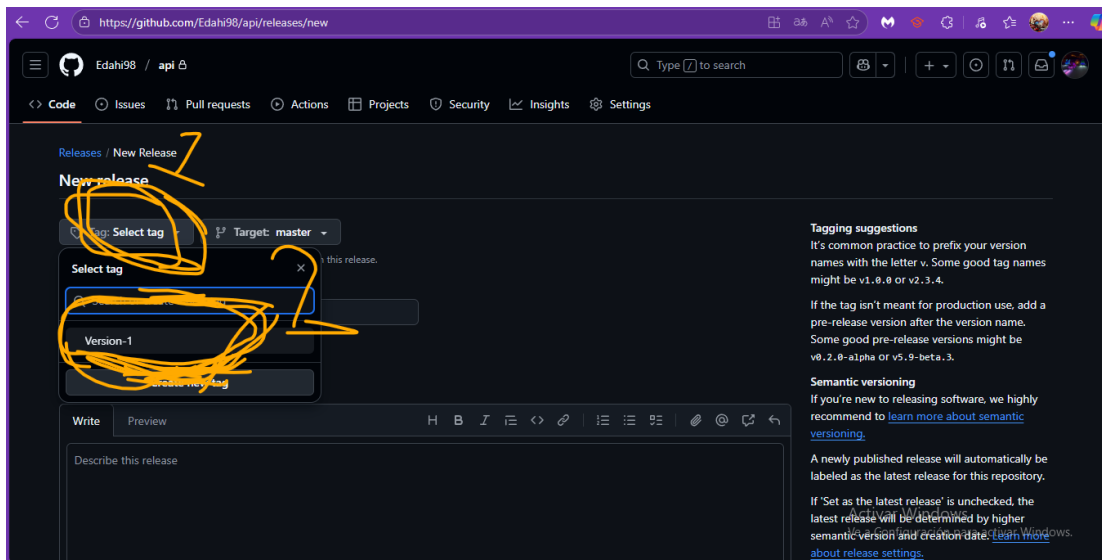
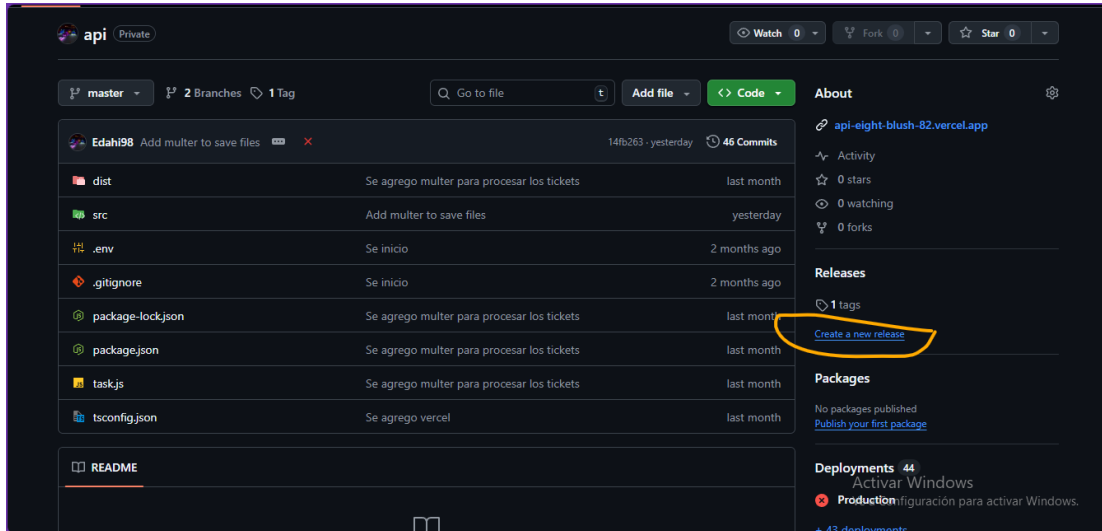
- 1) Ir a history
- 2) Hacer clic derecho en el commit deseado.
- 3) Hacer clic en "Create Tag"
- 4) Escribir el tag de acuerdo a la especificación de versionamiento semántico.

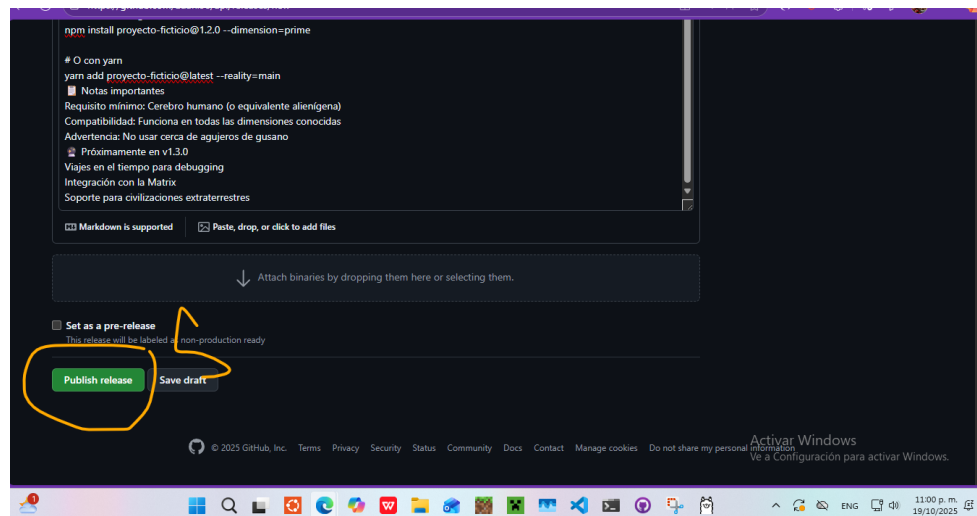
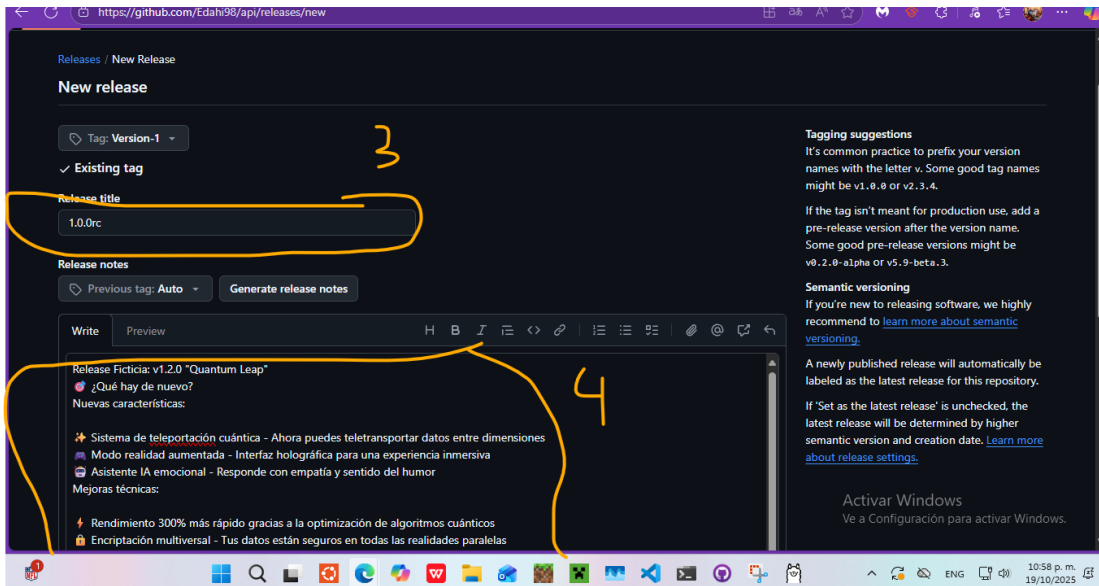


5) Finalmente, hacer clic en “Create Tag”.

Creación de una release:

Entra a la web de Github:





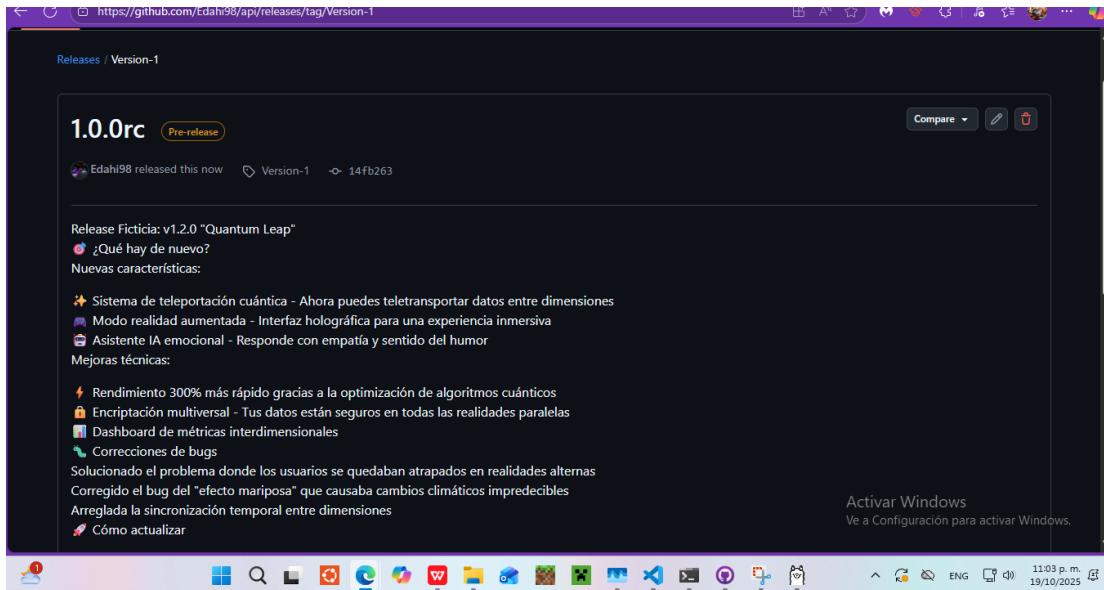
Entrar al repositorio y hacer clic en “Create a new release”

1. Hacer clic en “Select Tag”.
2. Seleccionar el tag
3. Escribir el titulo
4. Ingresa el contenido de la release
5. Finalmente, dar clic en “Publish Release”



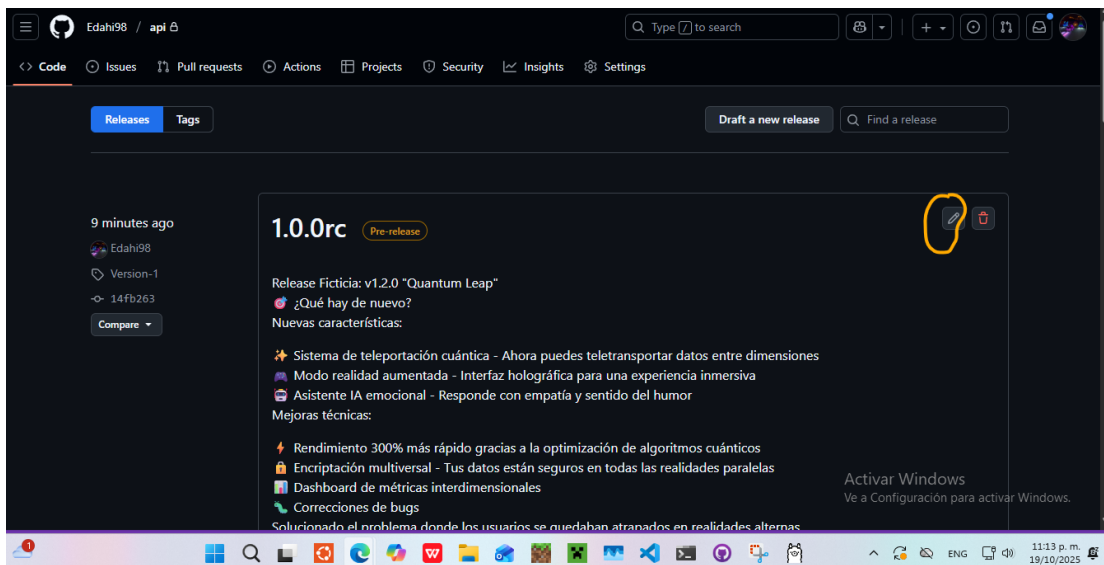
Reparaciones y venta de autopartes

Finalmente se mostrara de la siguiente forma:



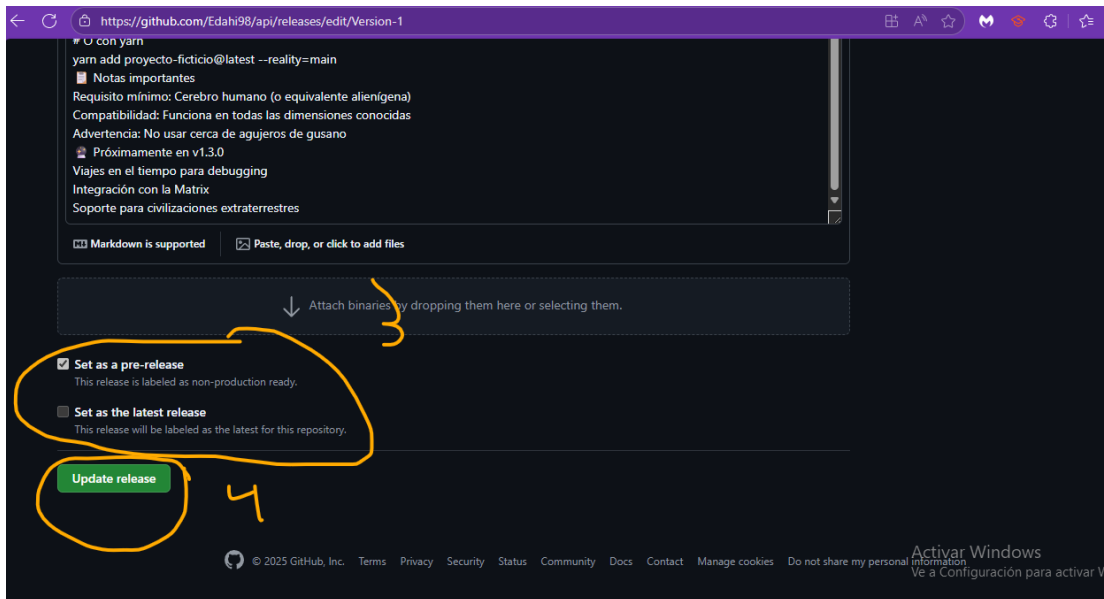
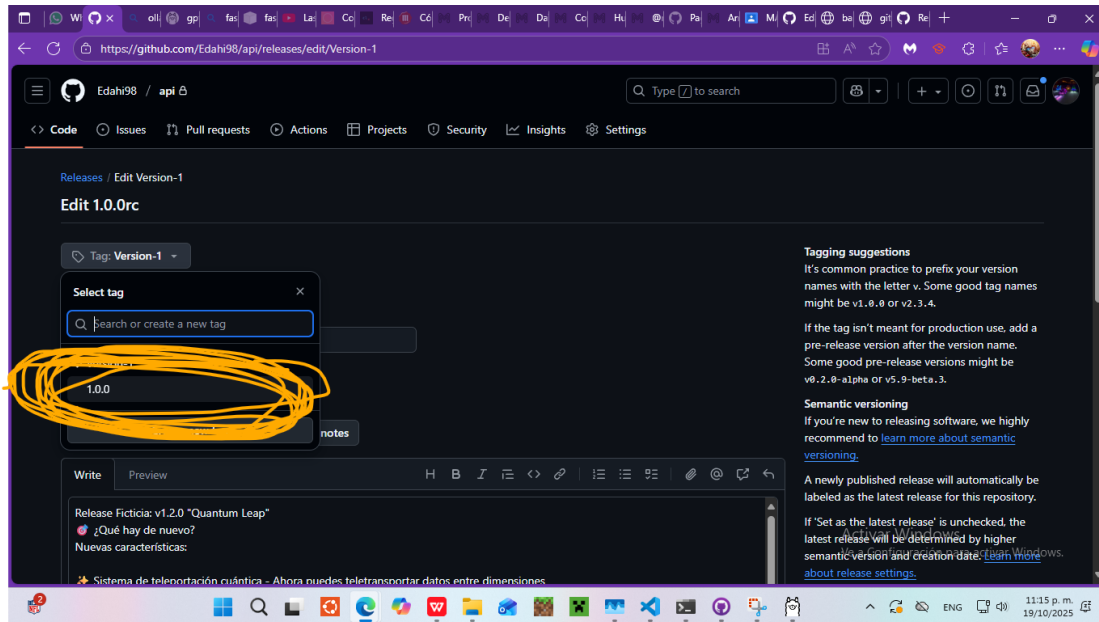
Gestionamiento de tags y releaseÑ

Editar





Reparaciones y venta de autopartes



- 1) Dar clic en el lápiz de editar
- 2) Seleccionar una nueva tag, para crear puedes seguir los mismos pasos de la parte de creación de tag.



- 3) Se cambia como se va a marcar la la branch release, si va a ser latest, pre-release, etc.
- 4) Guardar cambios.

BUENAS PRÁCTICAS

Commits

- Explica por qué y cómo se hizo, envuelto a 72 caracteres.
- Usa verbos en presente (“Fix bug with Y”), no pasivo (“Fixed bug”).
- ≤ 50 caracteres; el resto del mensaje puede ser más extenso.
- Cada commit debe contener una única unidad lógica de trabajo (ej. cambiar una variable, luego añadir pruebas en otro commit).

Ramas

- Trabaja siempre en una rama temporal creada a partir de main.
- Todos trabajan directamente sobre main (o trunk) con cambios muy pequeños y feature flags.
- main contiene siempre código en producción.
- Cada rama lleva un único objetivo (feature, bug-fix, hot-fix).
- Cuando la rama está lista, abre un Pull Request y, tras aprobación y CI, fusiona a main.

Merge Request

- Exige al menos una revisión aprobada.
- Limita el número de revisores.
- No añadas archivos binarios grandes.
- Borra la rama después del merge.

RECURSOS Y SOPORTE



Recursos

1. GitHub Flow (creación de ramas, commits, PR, merge)
 - a) <https://docs.github.com/en/get-started/using-github/github-flow>
2. Branch protection rules (revisiones obligatorias, status checks, firma de commits)
 - a) <https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>
3. Cómo crear y administrar Pull Requests
 - a) <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/about-pull-requests>
4. Resolución de conflictos en GitHub
 - a) <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github>
5. GitHub Actions (CI/CD)
 - a) <https://docs.github.com/en/actions>
6. Naming conventions for Git branches
 - a) <https://www.devopstraininginstitute.com/blog/what-are-the-best-practices-for-branch-naming-conventions-in-git-repositories>
7. Conventional Commits – mejores prácticas de mensaje
 - a) <https://graphite.dev/guides/git-commit-message-best-practices>
8. Tagging y versionado semántico
 - a) <https://blog.pixelfreestudio.com/best-practices-for-creating-and-using-git-tags/>
9. Automatizar el formateo con Prettier



- a) <https://www.codewalnut.com/tutorials/how-to-automate-code-formatting-with-prettier-using-github-actions>

10. GitHub Flow – introducción visual

- a) <https://docs.github.com/en/get-started/using-github/github-flow>

Soporte

- a) Slack - Github
b) Slack - Prettier
c) X - Prettier
d) X - Github

URL del repositorio: https://github.com/Edahi98/LosHermanos_Inventario

GESTIÓN DE VERSIONES

Versión	Responsable	Fecha
0.0	Edahi,Gachuz	3/11/2025
1.0	Edahi,Gachuz	13/11/2025
2.0	Edahi,Gachuz	20/11/2025
3.0	Edahi,Gachuz	27/11/2025

CERTIFICADO TLS COMO MEDIDA DE SEGURIDAD

Justificación



La necesidad de TLS se vuelve crítica, debido a la arquitectura de microservicios propuesta. La lógica sensible de la actualización de inventario reside en el microservicio de stock, que interactúa con los datos de la misma base de datos que ocupa la aplicación web.

- a) Vulnerabilidad de comunicación con el microservicio de stock
 - a) Una arquitectura MVC que ocupa microservicios crea un punto de interconexión donde el servicio debe tratarse como un posible punto de ataque, la web puede verse vulnerable. Un atacante podría explotar las llamadas a la API, manipulando directamente las cantidades de stock de las refacciones.
- b) Zero Trust
 - a) Se debe proteger el inventario de la manipulación externa, será requerido Mutual Transport Layer Security (mTLS). Esto se basa en el principio de seguridad Zero Trust, donde ningún servicio es confiable por defecto.

Evidencia

- a) Preparación de la Infraestructura de Certificados
 - a) Configuración
 - i. En el servidor Express, la función estándar `app.listen()` es reemplazada por `https.createServer()`, con los parámetros que obligan la autenticación de cliente.



```
1 // Configuración de mTLS para el Microservicio (TypeScript/Express)
2 const https = require('https');
3 const express = require('express');
4 const app = express();
5
6 //... (rutas y lógica)...
7
8 https.createServer( {
9   // 1. Identifica el Microservicio (Server)
10  cert: fs.readFileSync('server.crt'),
11  key: fs.readFileSync('server.key'),
12  // 2. EXIGE el certificado del cliente (Laravel)
13  requestCert: true,
14  // 3. Define la CA de confianza para validar el certificado de Laravel
15  ca: fs.readFileSync('ca.crt'),
16  rejectUnauthorized: false, // La autorización se maneja por middleware
17 }, app ).listen(9443);
```

b) Archivo: server.crt / server.key

- i. Identidad criptográfica del Microservicio Express: Usado por Express para demostrar su autenticidad a Laravel.

c) Archivo: ca.crt

- i. Certificado público: Para que ambas partes confíen en los certificados presentados por la contraparte.

MECANISMO DE AUTENTIFICACIÓN POR JSON WEB TOKEN DE MANERA REMOTA

Justificación

JSON Web Token (JWT) es un requisito en la arquitectura de microservicios propuesta y la naturaleza crítica de los datos manejados, que incluye la actualización de stock en tiempo real y el cálculo automático de impuestos (IVA, ISR). JWT no es solo un método de autenticación, sino un protocolo de seguridad que garantiza el Rendimiento.

Evidencia

Se utilizan las bibliotecas express-jwt y jwks-rsa para manejar la validación del token de forma externa y segura. El middleware checkJwt realiza los pasos de autenticación.

```
1 // src/middleware/checkJwt.middleware.ts
2 import { expressJwt, GetVerificationKey } from "express-jwt";
3 import jwksRsa from "jwks-rsa";
4 import * as dotenv from "dotenv";
5
6 dotenv.config();
7
8 export const checkJwt = expressJwt({
9   secret: jwksRsa.expressJwtSecret({
10     cache: true,
11     ratelimit: true,
12     jwksRequestsPerMinute: 5,
13     jwksUri: `https://${process.env.AUTH_DOMAIN}/.well-known/jwks.json`
14   }) as GetVerificationKey, // Tipado requerido para TS
15   audience: process.env.AUTH_AUDIENCE,
16   issuer: `https://${process.env.AUTH_DOMAIN}/`,
17   algorithms: [], // Algoritmo estándar
18   requestProperty: 'user', // Payload decodificado almacenado en req.user
19   getToken: (req) => req.headers.authorization?.split(" ")[1] // Obtener Bearer token
20 });
```

LARAVEL SANCTUM EN LA PARTE WEB (WEB SERVICE)

Justificación

Laravel Sanctum es la opción de autenticación más adecuada, porque está diseñado para proporcionar una solución de seguridad ligera, rápida y unificada que se ajusta perfectamente a las necesidades de una aplicación web de gestión.

Dado que el enfoque del sistema se centra en una aplicación web centralizada para gestionar el Inventario. Sanctum ofrece beneficios directos en la simplicidad de la implementación y la seguridad de la interfaz de usuario.

Evidencia



Laravel Sanctum se integraría para habilitar dos flujos de trabajo de autenticación esenciales para el sistema de gestión del taller: la experiencia de usuario dentro del navegador y el control de acceso a los datos mediante tokens API.

a) Habilitación de tokens en el modelo del usuario.

```
// app/Models/User.php
// app/Models/User.php
use Illuminate\Foundation\Auth\User as Authenticatable;
use Laravel\Sanctum\HasApiTokens; // <--- Trait esencial

class User extends Authenticatable
{
    use HasApiTokens; // habilita la gestión de tokens
    //... otros traits y código
}

// II. Evidencia de uso para autenticación web (SPA)
Dado que la interfaz de gestión (vistas de Inventario, Reportes, Servicios) funciona como una aplicación web [3], Sanctum utiliza la autenticación basada en sesiones y "cookies" de Laravel, que es la forma más segura de gestionar el acceso en un navegador.

// III. Configuración de Datos "Stateful"
Para garantizar la protección contra CSRF (Cross-Site Request Forgery) y el flujo de sesión, se debe configurar el dominio del sistema en el archivo de configuración de Sanctum.

// config/sanctum.php
'stateful' => true,
[4, 5]

Esta configuración lo dice a Laravel que monta la "sesión" para el personal interno a través de una "cookie" segura, sin depender de un token visible en el "frontend" [5].

// III. Evidencia de uso para Autenticación Fina (API/Token/Token)
La característica más importante de Sanctum es su capacidad de "autorización granular" para proteger los "endpoints" sensibles, como la modificación del stock o el precio, venta [1].

// III.1. Definición y Emisión de Tokens con Habilidades
Para aplicar el "Principio de Mínimo Privilegio (MPP)" [6], el código de Laravel permite generar tokens limitados en sus permisos.

* "Implementación de Endpoint de Creación de Token (Solo Admin)"
// routes/api.php
use Illuminate\Http\Request;
use App\Models\User;

Route::post('/tokens/create', function (Request $request) {
    // Verificar que el usuario esté autenticado por sesión
    $user = $request->user();

    // Generar un token con habilidades específicas (e.g., solo lectura de reportes)
    $token = $user->createToken([
        'permissions' => [
            // Habilidades/tokens específicas:
            'reportes.read', 'inventario.read'
        ], // [1]
    ]);

    // El token de texto plano solo se obtiene aquí y debe almacenarse de forma segura por el cliente.
    return response($token->plainTextToken, 201);
})->middleware('auth:sanctum');

// III.2. Protección de Rutas con "Middleware" de Habilidades
Una vez que los tokens se generan con "scopes" específicos, se utiliza el "middleware" de Sanctum para proteger las rutas críticas, asegurando que solo el personal autorizado pueda ejecutar acciones de alto riesgo.

* "Configuración del "Middleware": Los alias "abilities" y "ability" se registran en el "Service Provider" de Laravel [1].
* "Protección del Módulo de Inventario": El siguiente ejemplo de código demuestra cómo se protege la función de actualización de inventario (un CRUD crucial) [1].

Route::middleware(['auth:sanctum'])->group(function () {
    // 1. Acceso de LECTURA (Bajo Riesgo)
    // Se requiere la habilidad "inventario.read" @ "reportes.read"
    Route::get('/articulos', function () {
        // Accede al stock y los precios
    })->middleware('ability:inventario.read,reportes.read'); // [1]

    // 2. Acceso de ESCRITURA (Alto Riesgo: Modificación de Stock/Precios)
    // Se requiere al menos una de las habilidades "inventario.write"
    Route::put('/articulos/{id}', [ArticleController::class, 'update'])->
        middleware('ability:inventario.write'); // [1]

    // Si el token solo tiene "inventario.read", esta ruta devolverá un 403 Forbidden.
});

// La implementación de Sanctum a este nivel garantiza que el sistema cumple con los estándares de seguridad RESTful, utilizando "tokens" para proteger los "endpoints" de la API y aplicando una capa de autorización fina para salvaguardar los datos sensibles del inventario y la facturación.
```